NANYANG TECHNOLOGICAL UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Diffusion Models for Intelligent Image Editing and Inpainting

Lee Yu Quan

Supervisor: Prof Zhang Hanwang

School of Computer Science and Engineering

A Final Year Project report
presented to Nanyang Technological University
in partial fulfilment of the requirements for the
degree of Bachelor of Engineering

2025/2026

Acknowledgements

Over the course of two semesters working on this final year project, I would like to express my appreciation to everyone who has encouraged me and offered their guidance, helping make this project possible.

I would like to extend my sincere gratitude to my supervisor, Prof Zhang Hanwang, for granting me the freedom to steer the direction of this project. His trust and openness allowed me to explore a wide variety of diffusion models and techniques in the field of image generation, which greatly enriched both the project and my learning experience.

Lastly, I would like to thank my examiner, Prof (placeholder), for taking the time to review and evaluate this final year project.

Lee Yu Quan
March 2026

Contents

List of Figures

List of Tables

1 Introduction

1.1 Abstract

This report presents DiffusionDesk, a web-based image editing platform that leverages open-source diffusion models to provide intelligent inpainting, style transfer, and image restoration capabilities within a unified interface. The application addresses the fragmentation and complexity barriers of existing AI-powered image editing tools by integrating multiple state-of-the-art models—including Stable Diffusion, Stable Diffusion XL, Kandinsky 2.2, and FLUX.1 Fill for inpainting; SDXL image-to-image for style transfer; and CodeFormer, GFPGAN, and Real-ESRGAN for restoration—into a single, accessible web application. The system is built with a FastAPI backend serving model inference on GPU-equipped hardware and a React frontend providing an intuitive browser-based interface. Key technical contributions include a VRAM optimisation strategy using BitsAndBytes quantisation (4-bit and 8-bit) that reduces FLUX.1 Fill memory requirements from 22–24 GB to approximately 10 GB, an interactive canvas-based mask drawing component for precise inpainting control, and a modular service architecture that supports dynamic model loading and caching. The platform is designed for deployment on cloud GPU environments such as Google Colab, making diffusion-based image editing accessible to users without local GPU hardware or programming expertise.

1.2 Background and Motivation

Large language models are the most discussed aspect of generative AI today, but image generation is not far behind. According to Grand View Research, the global AI image generator market was valued at USD 349.6 million in 2023 and is projected to grow at a compound annual growth rate (CAGR) of 17.7% to reach USD 1.08 billion by 2030 (Grand View Research, 2023). Yet, there remains a lack of comprehensive software that caters to this growing demand in an accessible manner.

Traditional methods in image inpainting, such as patch-based and exemplar-based approaches, have notable limitations in generating semantically meaningful content, particularly in high-resolution or complex scenarios (Ma et al., 2023). These methods often struggle with boundary artefacts when dealing with large masked regions due to insufficient constraints, resulting in visible seams and structurally inconsistent outputs (Ma et al., 2023). Such limitations create significant accessibility barriers, as current solutions frequently require extensive technical expertise and expensive software licences, effectively restricting advanced image editing capabilities to professional users.

The introduction of deep learning techniques, particularly diffusion models (Ho et al., 2020; Song et al., 2021; Rombach et al., 2022), has led to significant improvements in image generation quality and semantic comprehension, enabling capabilities that were previously

difficult or impossible to automate. A detailed review of these developments is presented in Section 3.

Despite these breakthroughs, critical gaps persist between state-of-the-art image editing models and users' practical needs. Existing solutions face four key limitations: (1) fragmented ecosystems requiring users to switch between different applications for different editing tasks, (2) high complexity barriers that make advanced editing tools inaccessible to non-expert users, (3) reliance on command-line tools, Python programming, and GPU-equipped hardware, and (4) a lack of unified platforms that integrate multiple diffusion model capabilities within a single interface.

This project, DiffusionDesk, addresses these gaps by deploying a web-based platform that integrates diffusion models for inpainting, style transfer, and image restoration within a single, user-friendly interface. Developed as a Final Year Project at Nanyang Technological University under the supervision of Prof Zhang Hanwang, the application provides three core image editing capabilities:

1. **Inpainting** – removing or replacing objects within selected regions of an image using models such as Stable Diffusion, Stable Diffusion XL, Kandinsky, and FLUX.1 Fill.

2. **Style Transfer** – applying artistic styles such as anime, oil painting, and watercolour to images using diffusion-based image-to-image translation.

3. **Restoration** – enhancing image quality through face restoration (CodeFormer, GFPGAN) and image upscaling (Real-ESRGAN).

By exposing these models through a FastAPI backend and a React-based browser frontend, DiffusionDesk aims to make diffusion-based image editing accessible to users without requiring direct interaction with the underlying models or command-line tools.

1.3 Project Objective

The objective of this project is to design and deploy a web-based image editing platform that leverages open-source diffusion models to provide intelligent inpainting, style transfer, and image restoration capabilities. The specific objectives are as follows:

1. To develop a responsive web application that integrates multiple diffusion models for image editing within a unified interface.

2. To implement an inpainting feature that enables users to selectively remove or replace objects in images using state-of-the-art diffusion models, including Stable Diffusion, Stable Diffusion XL, Kandinsky, and FLUX.1 Fill.

3. To implement a style transfer feature that allows users to apply artistic styles to images through diffusion-based image-to-image translation.

4. To implement an image restoration feature that enhances image quality through face restoration and upscaling using CodeFormer, GFPGAN, and Real-ESRGAN.

5. To design an intuitive user interface that enables non-expert users to perform advanced image editing tasks without requiring technical expertise in machine learning or programming.

6. To evaluate the system's performance through processing speed benchmarks, output quality assessments, and usability considerations.

1.4 Limitations

This project is subject to the following limitations:

1. **Open-source models only** – The application exclusively utilises open-source diffusion models available through the Hugging Face ecosystem. Proprietary or commercially licensed models are not included, which may limit the range of available capabilities compared to commercial solutions.

2. **GPU resource constraints** – Diffusion model inference is computationally intensive and requires GPU acceleration. The available GPU memory (VRAM) constrains the size and complexity of models that can be loaded simultaneously. Quantisation techniques (4-bit, 8-bit) are employed to mitigate this, but may result in slight quality degradation.

3. **Supported image formats** – The application supports JPEG, JPG, and PNG image formats only. Other formats such as TIFF, BMP, WebP, or RAW are not supported.

4. **No mobile application** – The platform is designed as a web application accessible through desktop and mobile browsers. A dedicated native mobile application is not within the project scope.

5. **No video processing** – The system processes individual images only. Video frame processing, video inpainting, or video style transfer are not supported.

6. **No 3D image manipulation** – The application is limited to 2D image editing. 3D reconstruction, 3D-aware editing, or depth-based manipulation are not included.

7. **Inference only** – The project focuses on model inference using pre-trained models. Model training, fine-tuning, or custom model development are outside the project scope.

1.5 Project Scope

The scope of this project encompasses the following:

1.5.1 In Scope

- Development of a web-based frontend using React, TypeScript, and Tailwind CSS that provides an intuitive user interface for all three editing features.
- Development of a backend API using FastAPI and PyTorch that serves diffusion model inference for inpainting, style transfer, and image restoration.
- Implementation of inpainting using Stable Diffusion Inpainting, Stable Diffusion XL Inpainting, Kandinsky Inpainting, and FLUX.1 Fill models.
- Implementation of style transfer using SDXL image-to-image generation with artistic style prompts.
- Implementation of image restoration using CodeFormer, GFPGAN (face restoration), and Real-ESRGAN (image upscaling).
- Support for JPEG, JPG, and PNG image formats.
- VRAM optimisation through model quantisation (4-bit, 8-bit) and CPU offloading to accommodate varying GPU configurations.
- Deployment and testing on cloud GPU environments such as Google Colab.

1.5.2 Out of Scope

- Native mobile application development.
- Video processing, video inpainting, or video style transfer.
- 3D image manipulation or depth-based editing.
- Model training, fine-tuning, or custom model development.
- User authentication, user account management, or multi-user collaboration features.
- Image formats other than JPEG, JPG, and PNG.

Success will be measured through processing speed benchmarks, output quality assessments, and user experience evaluation across the three core features.

2 Project Schedule

This section outlines the project timeline, work breakdown structure, and risk management plan for DiffusionDesk. The project spans two semesters of Academic Year 2025/2026, with key milestones aligned to the FYP submission deadlines.

2.1 Project Timeline

Table 1 presents the key milestones and deliverables for the project.

Table 1: Project Timeline and Milestones

Date	**Week**	**Milestone**
11 Aug 2025	Sem 1, Wk 1	FYP officially commences
1 Sep 2025	Sem 1, Wk 4	Submission of Project Plan to Supervisor
Oct 2025	Sem 1, Wk 8–10	Backend API and diffusion service implementation
Nov 2025	Sem 1, Wk 12–13	Frontend development and API integration
26 Jan 2026	Sem 2, Wk 3	Submission of Interim Report
Feb 2026	Sem 2, Wk 5–7	Feature refinement and testing on cloud GPU
23 Mar 2026	Sem 2, Wk 10	Submission of Final Report
17 Apr 2026	Sem 2, Wk 13	Submission of Amended Final Report
8–13 May 2026	–	Oral Presentation (20 min + 10 min Q&A)

2.2 Work Breakdown

This section provides a structured breakdown of the project activities, their descriptions, estimated effort, and dependencies. The work breakdown structure facilitates progress tracking and resource allocation throughout the development lifecycle.

Table 2: Work Breakdown Structure

ID	**Activity**	**Description**	**Effort (Days)**	**Depends On**
1.1	Literature Review	Review foundational papers on diffusion models (DDPM, DDIM, LDM) and existing inpainting, style transfer, and restoration techniques.	10	–

ID	Activity	Description	Effort (Days)	Depends On
1.2	Technology Evaluation	Evaluate and select appropriate frameworks, libraries, and pre-trained models from the Hugging Face ecosystem.	5	1.1
1.3	Requirements Specification	Define functional and non-functional requirements based on project objectives and supervisor feedback.	4	1.2
2.1	Backend Architecture Design	Design the FastAPI backend structure, including API endpoints, service layers, and model management strategy.	6	1.3
2.2	Inpainting Service	Implement the inpainting service supporting SD, SDXL, Kandinsky, and FLUX.1 Fill models with quantisation support.	15	2.1
2.3	Style Transfer Service	Implement the style transfer service using SDXL img2img with configurable style prompts and parameters.	10	2.1
2.4	Restoration Service	Implement face restoration (CodeFormer, GFPGAN) and image upscaling (Real-ESRGAN) services.	10	2.1
2.5	VRAM Optimisation	Implement model quantisation (4-bit, 8-bit) and CPU offloading strategies to support varying GPU configurations.	8	2.2, 2.3
3.1	Frontend UI Design	Design the user interface layout, including wireframes for the Home, Inpainting, Style Transfer, and Restoration tabs.	5	1.3
3.2	Frontend Implementation	Develop the React frontend with TypeScript and Tailwind CSS, implementing all UI components and tab navigation.	15	3.1
3.3	Canvas and Mask Drawing	Implement the interactive canvas component for users to draw inpainting masks on uploaded images.	8	3.2

ID	Activity	Description	Effort (Days)	Depends On
3.4	API Integration	Connect the frontend to the backend API, implementing image upload, processing requests, and result display.	6	2.2–2.4, 3.2
4.1	Cloud Deployment Testing	Deploy and test the backend on Google Colab with ngrok tunnelling to validate GPU inference performance.	5	3.4
4.2	Feature Testing	Conduct end-to-end testing of all features, addressing bugs and refining based on test results.	10	4.1
4.3	Performance Benchmarking	Measure and document inference times, memory usage, and output quality across different models.	5	4.2
5.1	Supervisor Meetings	Regular meetings with the supervisor to report progress, seek guidance, and align on project direction.	8	All
5.2	Interim Report Writing	Prepare and submit the interim report documenting progress, challenges, and preliminary results.	5	2.5, 3.4
5.3	Final Report Writing	Prepare the final report with comprehensive documentation of system design, implementation, and evaluation.	15	4.3, 5.2
5.4	Oral Presentation Prep	Prepare presentation slides and rehearse for the oral examination.	5	5.3

2.3 Risk Management

This section identifies potential risks that may impact the project's success and outlines mitigation strategies. Each risk is assessed based on its probability of occurrence, potential impact, and overall risk level. Proactive risk management ensures that challenges are anticipated and addressed promptly.

Table 3: Risk Management Plan

Risk	**Mitigation Strategy**	**Prob.**	**Impact**	**Level**
Insufficient GPU memory for large models	Implement model quantisation (4-bit, 8-bit) using bitsandbytes and enable CPU offloading. Prioritise smaller models when VRAM is limited.	High	High	High
Slow model inference	Use DDIM sampling with reduced steps (20–50), enable attention slicing, and implement model caching to reduce loading overhead.	Medium	High	Medium
Low-quality or artefacted outputs	Fine-tune prompt engineering, adjust guidance scale and denoising strength, and provide users with parameter controls for iterative refinement.	Medium	Medium	Medium
Diffusers library breaking changes	Pin specific library versions in requirements.txt and test compatibility before updating dependencies.	Medium	Medium	Medium
Cloud GPU constraints (Colab limits)	Design for stateless operation, allowing sessions to restart without data loss. Document alternative deployment options (NTU HPC, local GPU).	Medium	Medium	Medium
Frontend-backend integration issues	Define clear API contracts using OpenAPI, implement comprehensive error handling, and test integration incrementally.	Medium	High	Medium
Scope creep	Adhere strictly to defined scope. Evaluate new requests against timeline constraints and prioritise core features.	Medium	Medium	Medium
Unfamiliarity with diffusion architectures	Conduct thorough literature review early. Leverage tutorials, documentation, and pre-trained models to accelerate learning.	Low	High	Low
Time management challenges	Maintain detailed project schedule with milestones. Allocate buffer time and communicate proactively with supervisor.	Low	High	Low

3 Literature Review

3.1 Diffusion Models

The foundation of modern image generation lies in diffusion models, which produce images through an iterative denoising process. This subsection reviews the key developments that underpin the models used in this project.

3.1.1 Denoising Diffusion Probabilistic Models (DDPM)

Ho et al. (2020) proposed Denoising Diffusion Probabilistic Models (DDPMs), which generate images by treating the process as a series of denoising steps grounded in nonequilibrium thermodynamics. The forward process gradually adds Gaussian noise to an image over T timesteps until the image becomes pure noise. The reverse process then learns to denoise step by step, recovering a clean image from random noise. DDPMs demonstrated image quality that surpassed the then-dominant Generative Adversarial Networks (GANs), producing diverse, high-fidelity samples without the training instability commonly associated with GANs. However, the original DDPM formulation required a large number of denoising steps (typically $T = 1000$), resulting in slow sampling speeds.

3.1.2 Denoising Diffusion Implicit Models (DDIM)

Song et al. (2021) addressed the slow sampling limitation of DDPMs by proposing Denoising Diffusion Implicit Models (DDIMs). DDIMs reformulate the reverse diffusion process as a non-Markovian process, meaning that each denoising step can depend on the original noisy input rather than solely on the immediately preceding step. This reformulation allows for deterministic sampling and, crucially, enables the use of a subsequence of only 20–100 steps while maintaining comparable image quality. The result is a 10–50$\times$ speedup over DDPMs, making diffusion-based generation significantly more practical for interactive applications.

3.1.3 Latent Diffusion Models (LDM)

Rombach et al. (2022) proposed Latent Diffusion Models (LDMs), which achieved an optimal balance between generative quality and computational efficiency. Rather than performing diffusion directly in pixel space, LDMs first encode images into a lower-dimensional latent representation using a pre-trained autoencoder, then apply the diffusion process within this compressed latent space. This approach alleviates critical computational bottlenecks, substantially reducing memory and computation requirements while preserving high image quality. LDMs form the basis of the widely adopted Stable Diffusion family of models, including the inpainting and image-to-image variants used in this project.

3.1.4 Enabling Technologies for Conditional Generation

The diffusion models reviewed above provide the foundational denoising mechanism, but practical text-to-image systems require additional components for conditional generation. This subsection reviews three key enabling technologies that bridge the gap between unconditional diffusion and controllable image synthesis.

3.1.4.1 Classifier-Free Guidance Ho and Salimans (2022) introduced classifier-free guidance, a technique that enables conditional diffusion models to produce outputs strongly aligned with input conditions (e.g., text prompts) without requiring a separate classifier network. The method trains a single neural network to perform both conditional and unconditional generation by randomly dropping the conditioning signal during training. At inference time, the model's output is extrapolated away from the unconditional prediction towards the conditional prediction, controlled by a guidance scale parameter w:

$$\tilde{\epsilon}_\theta(z_t, c) = \epsilon_\theta(z_t, \emptyset) + w \cdot (\epsilon_\theta(z_t, c) - \epsilon_\theta(z_t, \emptyset)) \tag{1}$$

where $\epsilon_\theta(z_t, c)$ is the conditional prediction, $\epsilon_\theta(z_t, \emptyset)$ is the unconditional prediction, and w is the guidance scale. Higher values of w produce outputs more faithful to the conditioning but may reduce diversity and introduce artefacts. This parameter is exposed in DiffusionDesk as "guidance scale" and is fundamental to all inpainting and style transfer operations.

3.1.4.2 CLIP Text-Image Alignment Radford et al. (2021) developed CLIP (Contrastive Language-Image Pre-training), a model trained on 400 million image-text pairs to learn a joint embedding space for images and text. CLIP's text encoder transforms natural language prompts into dense vector representations that can guide image generation. In Stable Diffusion and similar models, the CLIP text encoder processes user prompts into conditioning embeddings that direct the denoising process. This architecture enables intuitive text-based control: users describe their desired output in natural language, and the model generates images aligned with that description. SDXL extends this by using dual text encoders (CLIP ViT-L and OpenCLIP ViT-G) for richer text understanding.

3.1.4.3 Diffusion Transformers (DiT) While Stable Diffusion and SDXL use U-Net architectures as their denoising backbone, Peebles and Xie (2023) demonstrated that Vision Transformers can serve as effective diffusion backbones. Diffusion Transformers (DiT) replace the convolutional U-Net with a transformer architecture, operating on sequences of latent patches. This approach scales more efficiently with model size and has led to state-of-the-art image generation quality. FLUX.1, developed by Black Forest Labs, adopts a DiT-based architecture with flow matching (a generalisation of diffusion that learns direct probability paths rather than score functions), achieving superior quality at the cost of increased computational requirements.

The architectural shift from U-Net to DiT represents a significant evolution in diffusion model design.

Having established the theoretical foundations—from DDPM's denoising mechanism, through DDIM's accelerated sampling, to LDM's latent space operation and the enabling technologies of classifier-free guidance, CLIP conditioning, and transformer backbones—the following sections examine how these principles are instantiated in practical models for specific image editing tasks.

3.2 Diffusion-Based Image Inpainting

Image inpainting is the task of filling in missing or masked regions of an image with plausible content. Traditional approaches include patch-based methods that copy similar patches from elsewhere in the image, and exemplar-based techniques that iteratively fill regions based on surrounding texture and structure. While effective for simple cases, these methods struggle with large masked regions, complex scenes, and semantically meaningful content generation—they cannot, for example, intelligently fill a masked region with a contextually appropriate object.

Deep learning approaches, particularly diffusion models, have transformed inpainting by learning rich semantic priors from large datasets. Rather than relying on local texture matching, diffusion-based inpainting models understand scene context and can generate novel content that is both visually coherent and semantically meaningful. This subsection reviews four diffusion-based inpainting models implemented in DiffusionDesk, each building upon the theoretical foundations established in Section 3.1.

3.2.1 Problem Definition

Given an input image x and a binary mask m indicating the region to be inpainted, the goal is to generate an output image $\hat{x}$ where the masked region contains plausible content consistent with the surrounding context. In diffusion-based inpainting, the model is additionally conditioned on a text prompt c that guides the generation. The inpainting process can be formulated as:

$$\hat{x} = f(x, m, c; \theta) \tag{2}$$

where f is the inpainting model with parameters θ. The model must preserve the unmasked regions exactly while generating coherent content in the masked areas.

3.2.2 Inpainting Model Architectures

3.2.2.1 Stable Diffusion Inpainting Stable Diffusion Inpainting (Rombach et al., 2022) extends the base Stable Diffusion v1.5 model for inpainting by modifying the input architecture. While the original model accepts a 4-channel latent input, the inpainting variant accepts 9

channels: 4 for the noisy latent, 4 for the encoded masked image, and 1 for the downsampled mask. This architectural modification allows the model to explicitly condition on both the masked image content and the mask geometry.

The model inherits the core DDPM/DDIM denoising mechanism and operates in the latent space as per LDM principles. Text conditioning is provided through the CLIP ViT-L/14 text encoder, and classifier-free guidance controls the fidelity to the text prompt. With approximately 860 million parameters and VRAM requirements of 5–7 GB in FP16 precision, Stable Diffusion Inpainting offers a good balance of quality and computational efficiency, making it suitable for users with limited GPU resources.

3.2.2.2 Stable Diffusion XL Inpainting SDXL Inpainting builds upon the base SDXL architecture, which introduced several improvements over Stable Diffusion v1.5: a larger U-Net backbone with approximately 2.6 billion parameters, native 1024×1024 resolution support, and dual text encoders (CLIP ViT-L and OpenCLIP ViT-G) for enhanced prompt understanding. These improvements translate to higher-quality inpainting results with better detail preservation and more accurate text-to-image alignment.

The theoretical foundations remain consistent: SDXL Inpainting uses DDPM-based denoising, supports DDIM and other accelerated schedulers, operates in latent space, and employs classifier-free guidance. However, the increased model capacity comes at a computational cost—SDXL Inpainting requires 10–12 GB VRAM in FP16 precision. To address this, DiffusionDesk implements 8-bit and 4-bit quantisation using the bitsandbytes library, reducing VRAM requirements to approximately 6 GB and 4 GB respectively, with minimal quality degradation.

3.2.2.3 Kandinsky 2.2 Inpainting Kandinsky 2.2 employs a distinct two-stage architecture inspired by unCLIP/DALL-E 2. The first stage ("prior") maps the text prompt to a CLIP image embedding, predicting what the image embedding should look like given the text. The second stage ("decoder") generates the actual image conditioned on both the text and the predicted image embedding. This architecture leverages CLIP's powerful joint text-image space to guide generation.

For inpainting, Kandinsky conditions the decoder on the masked image and mask geometry in addition to the CLIP embeddings. With approximately 2 billion parameters across both stages and VRAM requirements of 6–8 GB, Kandinsky offers comparable resource usage to Stable Diffusion while providing different aesthetic characteristics. The two-stage approach can produce results with strong text alignment due to the explicit CLIP image embedding conditioning.

3.2.2.4 FLUX.1 Fill FLUX.1 Fill, developed by Black Forest Labs, represents a significant architectural departure from the U-Net-based models. Built on the Diffusion Transformer

(DiT) architecture, FLUX.1 uses a transformer backbone operating on sequences of latent patches rather than a convolutional U-Net. Additionally, FLUX.1 employs flow matching rather than traditional score-based diffusion, learning direct probability paths between noise and data distributions.

With approximately 12 billion parameters and a T5-XXL text encoder (rather than CLIP), FLUX.1 Fill achieves state-of-the-art inpainting quality with exceptional detail, coherence, and prompt following. However, this quality comes at significant computational cost: full BF16 inference requires 22–24 GB VRAM. DiffusionDesk implements NF4 quantisation to reduce this to approximately 10 GB, enabling deployment on consumer GPUs like the NVIDIA T4 available in Google Colab.

3.2.3 Model Comparison

Table 4 summarises the key characteristics of the four inpainting models implemented in DiffusionDesk.

Table 4: Comparison of Inpainting Models

Criteria	**SD Inpainting**	**SDXL Inpainting**	**Kandinsky 2.2**	**FLUX.1 Fill**
Architecture	U-Net (LDM)	U-Net (LDM)	Prior + Decoder	DiT (Transformer)
Parameters	∼860M	∼2.6B	∼2B	∼12B
Text Encoder	CLIP ViT-L	CLIP + OpenCLIP	CLIP	T5-XXL
VRAM (FP16)	5–7 GB	10–12 GB	6–8 GB	22–24 GB
VRAM (4-bit)	N/A	∼4 GB	N/A	∼10 GB
Quality	Good	Very Good	Good	Excellent
Theory Basis	DDPM, DDIM, LDM, CFG	DDPM, DDIM, LDM, CFG	DDPM, DDIM, LDM, CLIP	Flow Matching, DiT

3.2.4 Selection Justification

DiffusionDesk includes all four inpainting models to provide users with flexibility across different use cases and hardware constraints:

- **SD Inpainting**: Entry-level option for users with limited VRAM (<8 GB). Provides good quality results with fast inference.
- **SDXL Inpainting**: Recommended default for users with mid-range GPUs (8–12 GB). Offers improved quality and prompt understanding with quantisation options for constrained environments.

- **Kandinsky 2.2**: Alternative architecture providing different aesthetic characteristics. Useful when SDXL results are unsatisfactory for specific prompts.
- **FLUX.1 Fill**: State-of-the-art quality for users with high-end GPUs or when quantised deployment is acceptable. Best choice for complex inpainting tasks requiring maximum coherence.

The diffusion architectures examined for inpainting can also be applied to style transfer. Rather than filling masked regions, style transfer leverages the same denoising process to transform an entire image's aesthetic while preserving its structure.

3.3 Diffusion-Based Style Transfer

Style transfer aims to render an image in a different artistic style while preserving its underlying content and structure. This section reviews the evolution of neural style transfer techniques and explains how diffusion models provide a powerful and flexible approach through the image-to-image (img2img) mechanism.

3.3.1 Evolution of Neural Style Transfer

Gatys et al. (2016) pioneered neural style transfer by demonstrating that convolutional neural networks (CNNs) encode both content and style information in their hierarchical feature representations. Their optimisation-based approach iteratively modifies an image to match the content features of a source image and the style features (captured via Gram matrices) of a reference style image. While producing impressive results, this method requires a slow optimisation process for each image pair.

Subsequent work developed feed-forward style transfer networks that train a single network per style, enabling real-time inference. However, these methods are limited to pre-defined styles and cannot generalise to arbitrary style descriptions. More recent approaches explored arbitrary style transfer using adaptive instance normalisation or attention mechanisms, but these often struggle with complex style semantics or structural preservation.

Diffusion models offer a compelling alternative: rather than explicitly separating content and style features, they leverage the denoising process with text conditioning to apply stylistic transformations described in natural language. This approach supports arbitrary styles without retraining and benefits from the semantic understanding encoded in large-scale text-image models.

3.3.2 Image-to-Image Mechanism

Diffusion-based style transfer operates through the image-to-image (img2img) pipeline, which differs fundamentally from text-to-image generation. Rather than starting from pure random

noise, img2img begins by adding controlled noise to the input image and then denoises with a style-describing prompt. This process can be understood as follows:

1. **Encoding**: The input image is encoded into latent space using the VAE encoder.
2. **Noise Addition**: Gaussian noise is added to the latent representation according to a specified "denoising strength" parameter $s \in [0, 1]$. Higher values add more noise, starting the denoising from a later timestep.
3. **Conditional Denoising**: The noised latent is denoised using the diffusion model, conditioned on a text prompt describing the desired style (e.g., "oil painting style", "anime illustration").
4. **Decoding**: The denoised latent is decoded back to pixel space.

The denoising strength parameter directly controls the trade-off between structure preservation and style application:

- **Low strength (0.2–0.4)**: Subtle stylisation; original composition, shapes, and details are largely preserved.
- **Medium strength (0.5–0.7)**: Moderate stylisation; structural elements remain recognisable but undergo significant aesthetic transformation.
- **High strength (0.8–1.0)**: Aggressive stylisation; the output may diverge substantially from the input, using it primarily as compositional guidance.

This mechanism directly leverages the DDPM/DDIM denoising process: the noise schedule determines at which timestep the denoising begins, and classifier-free guidance ensures the output adheres to the style prompt.

3.3.3 Approach Selection

DiffusionDesk implements style transfer using SDXL img2img for the following reasons:

- **Quality**: SDXL's larger capacity and dual text encoders produce higher-quality stylisations with better detail and prompt following compared to SD 1.5.
- **Resolution**: Native 1024×1024 support enables high-resolution style transfer without tiling artefacts.
- **Flexibility**: Text-based style conditioning supports arbitrary styles (anime, oil painting, watercolour, pencil sketch, cyberpunk, etc.) without requiring style reference images.
- **Parameter Control**: Exposing denoising strength, guidance scale, and inference steps gives users fine-grained control over the structure-style trade-off.

- **Quantisation Support**: 8-bit and 4-bit quantisation via bitsandbytes enables deployment on resource-constrained GPUs.

DiffusionDesk provides preset style prompts (anime, oil painting, watercolour, pencil sketch, digital art) while allowing users to specify custom prompts for specialised styles.

While diffusion models excel at generative tasks like inpainting and style transfer, image restoration requires specialised architectures optimised for specific degradation types. The following section examines restoration models that complement diffusion-based editing.

3.4 Image Restoration

Image restoration addresses the inverse problem of recovering high-quality images from degraded observations. Common degradations include compression artefacts, noise, blur, low resolution, and facial damage (wrinkles, scratches, low quality). Unlike the generative tasks of inpainting and style transfer, restoration aims to recover or enhance existing content rather than synthesise new content.

Notably, the restoration models reviewed in this section are *not* diffusion models. They employ distinct architectures—codebook-based transformers, generative adversarial networks (GANs), and enhanced super-resolution networks—each optimised for specific restoration tasks. DiffusionDesk includes these models to provide a comprehensive image editing pipeline that addresses both generative and restorative user needs.

3.4.1 Face Restoration

Face restoration is a specialised task that recovers high-quality facial details from degraded face images. The challenge lies in reconstructing plausible facial features (eyes, nose, mouth) while maintaining the subject's identity. DiffusionDesk implements two complementary approaches: CodeFormer and GFPGAN.

3.4.1.1 CodeFormer Zhou et al. (2022) proposed CodeFormer, a transformer-based face restoration method that leverages a learned discrete codebook of high-quality facial features. The approach consists of three components:

1. **Codebook Learning**: A vector-quantised autoencoder learns a discrete codebook capturing diverse high-quality facial features from a large face dataset.

2. **Code Prediction**: Given a degraded face image, a transformer predicts the sequence of codebook indices that best represent the underlying face.

3. **Controllable Decoding**: The decoder reconstructs the face from the predicted codes, with a fidelity parameter $w \in [0, 1]$ controlling the trade-off between restoration quality and identity preservation.

The fidelity parameter is particularly valuable: setting $w = 0$ produces maximum restoration quality (potentially altering facial features), while $w = 1$ maximally preserves the input identity (with less aggressive restoration). Users can adjust this parameter based on whether they prioritise visual quality or identity fidelity.

CodeFormer excels at restoring severely degraded faces, producing natural-looking results with realistic textures. It handles diverse degradation types robustly and provides controllable output through the fidelity parameter.

3.4.1.2 GFPGAN Wang et al. (2021a) developed GFPGAN (Generative Facial Prior GAN), which incorporates rich facial priors from a pre-trained face generation model (StyleGAN2) to assist face restoration. The key innovations include:

1. **Generative Facial Prior**: Channel-split spatial feature transforms inject pre-trained StyleGAN2 features to provide high-quality facial priors.

2. **Facial Component Dictionaries**: Pre-computed dictionaries of facial components (left eye, right eye, mouth) enable component-specific enhancement.

3. **Identity-Preserving Loss**: An identity loss term encourages the restored face to match the input identity.

GFPGAN tends to produce sharper results than CodeFormer in some cases but may occasionally alter facial features more aggressively. It performs particularly well on moderately degraded faces and old photographs.

3.4.1.3 Face Restoration Comparison Table 5 compares CodeFormer and GFPGAN across key characteristics.

Table 5: Comparison of Face Restoration Models

Criteria	**CodeFormer**	**GFPGAN**
Architecture	Transformer + Codebook	GAN + StyleGAN2 Prior
Controllability	Fidelity parameter (0–1)	Fixed
Severe Degradation	Excellent	Good
Identity Preservation	Controllable	Moderate
Output Sharpness	Natural	Sharp
VRAM Usage	2–4 GB	2–4 GB

DiffusionDesk includes both models to accommodate different user preferences and degradation scenarios. CodeFormer is recommended for severely degraded images or when identity preservation is critical, while GFPGAN is suitable for general enhancement of moderately degraded faces.

3.4.2 Image Upscaling

Image upscaling (super-resolution) increases image resolution while adding plausible high-frequency details. Classical interpolation methods (bicubic, bilinear) produce blurry results, while deep learning approaches can hallucinate realistic details.

3.4.2.1 Real-ESRGAN Wang et al. (2021b) extended ESRGAN (Enhanced Super-Resolution GAN) to handle real-world degradations. Unlike prior methods trained on synthetic bicubic downsampling, Real-ESRGAN models a comprehensive degradation pipeline including blur, noise, compression, and their combinations. This training strategy enables robust performance on diverse real-world images.

Key characteristics of Real-ESRGAN include:

- **RRDB Architecture**: Residual-in-Residual Dense Blocks provide powerful feature extraction for upscaling.
- **Real-World Degradation Model**: Training includes blur kernels, noise injection, JPEG compression, and resize operations in various orders.
- **Scale Options**: Supports $2\times$ and $4\times$ upscaling with pre-trained models.
- **Face Enhancement Integration**: A face-enhanced variant (Real-ESRGAN + GFPGAN) applies face restoration after upscaling for improved facial detail.

Real-ESRGAN produces sharp, detailed upscaled images with minimal artefacts, making it suitable for enhancing low-resolution photographs, enlarging images for printing, or improving image quality before further editing.

3.4.3 Selection Justification

DiffusionDesk includes CodeFormer, GFPGAN, and Real-ESRGAN to provide comprehensive restoration capabilities:

- **CodeFormer**: Primary face restoration model offering controllable quality-fidelity trade-off.
- **GFPGAN**: Alternative face restoration for users preferring sharper outputs or as a fallback when CodeFormer results are unsatisfactory.
- **Real-ESRGAN**: Essential for image upscaling, complementing face restoration for full-image enhancement pipelines.

These restoration models integrate seamlessly with diffusion-based editing: users can upscale low-resolution images before inpainting, or restore faces after style transfer to recover facial details.

3.5 Technology Stack

This section reviews and justifies the technology choices for developing DiffusionDesk. Each subsection follows a structured approach: exploring available options, comparing them against relevant criteria, and justifying the final selection.

3.5.1 Machine Learning Framework Selection

3.5.1.1 Options Explored Two major deep learning frameworks were considered for model inference:

- **PyTorch**: Developed by Meta AI, known for dynamic computation graphs, Pythonic API, and strong research community adoption.
- **TensorFlow**: Developed by Google, known for production deployment tools, static graph optimisation, and TensorFlow Serving.

Table 6: ML Framework Comparison

Criteria	**PyTorch**	**TensorFlow**
Diffusion Model Support	Excellent (Diffusers, native)	Limited (some ports exist)
Pre-trained Models	Extensive (Hugging Face Hub)	Moderate
Dynamic Graphs	Native	TF2 eager mode (added later)
Debugging	Straightforward (Python)	More complex (graph mode)
Research Adoption	Dominant in generative AI	Strong in production ML
Quantisation Libraries	bitsandbytes, GPTQ, AWQ	TensorFlow Lite

3.5.1.2 Comparison

3.5.1.3 Decision PyTorch was selected as the ML framework for the following reasons:

- **Diffusers Library**: The Hugging Face Diffusers library, which provides unified pipeline abstractions for all diffusion models used in this project, is built on PyTorch.
- **Model Availability**: All target models (SD, SDXL, Kandinsky, FLUX.1, CodeFormer, GFPGAN, Real-ESRGAN) have official PyTorch implementations available on Hugging Face Hub.

- **Quantisation Support**: The bitsandbytes library for 8-bit and 4-bit quantisation is PyTorch-native and essential for deploying large models on resource-constrained GPUs.
- **Research Alignment**: PyTorch dominates generative AI research, ensuring access to the latest models and techniques.

3.5.1.4 Supporting Libraries The following PyTorch ecosystem libraries are utilised:

- **Diffusers**: Unified pipeline API for diffusion models, scheduler implementations (DDIM, DPM++, Euler), and model loading utilities.
- **Transformers**: Text encoder loading (CLIP, T5) for conditioning diffusion models.
- **bitsandbytes**: 8-bit (LLM.int8) and 4-bit (NF4) quantisation for reduced VRAM usage.
- **Accelerate**: Device placement and mixed-precision inference utilities.

3.5.2 Backend Framework Selection

3.5.2.1 Options Explored Three Python web frameworks were considered for the API backend:

- **FastAPI**: Modern, async-first framework with automatic OpenAPI documentation.
- **Flask**: Lightweight, mature framework with extensive ecosystem.
- **Django**: Full-featured framework with ORM, admin interface, and batteries-included philosophy.

Table 7: Backend Framework Comparison

Criteria	**FastAPI**	**Flask**	**Django**
Async Support	Native (ASGI)	Extension (async views)	Limited (ASGI adapter)
Type Hints	Native (Pydantic)	Manual	Manual
Auto Documentation	OpenAPI + Swagger UI	Extension required	Extension required
Performance	High (async I/O)	Moderate	Moderate
Learning Curve	Low	Low	Moderate
ML Ecosystem Fit	Excellent	Good	Moderate (ORM overhead)

3.5.2.2 Comparison

3.5.2.3 Decision

FastAPI was selected for the following reasons:

- **Async Support**: Native async/await enables efficient handling of long-running inference requests without blocking other clients.
- **Pydantic Integration**: Type-validated request/response models ensure robust API contracts and clear documentation.
- **Automatic Documentation**: Built-in Swagger UI and OpenAPI specification generation facilitates frontend integration and testing.
- **Performance**: ASGI-based architecture provides high throughput suitable for inference workloads.
- **Simplicity**: No unnecessary features (ORM, admin) that add complexity without benefit for an ML inference API.

3.5.3 Frontend Framework Selection

3.5.3.1 Options Explored

Three major frontend frameworks were considered:

- **React**: Component-based library by Meta with extensive ecosystem.
- **Vue**: Progressive framework with gentle learning curve.
- **Angular**: Full-featured framework by Google with strong typing.

Table 8: Frontend Framework Comparison

Criteria	**React**	**Vue**	**Angular**
TypeScript Support	Excellent	Good	Native
Component Model	Functional + Hooks	Options/Composition API	Class-based
Ecosystem Size	Largest	Large	Large
Learning Curve	Moderate	Low	Steep
Canvas Libraries	Extensive (Fabric.js, Konva)	Good	Limited
Build Tooling	Vite, Next.js, CRA	Vite, Nuxt	Angular CLI

3.5.3.2 Comparison

3.5.3.3 Decision

React with TypeScript was selected for the following reasons:

- **Ecosystem**: The largest ecosystem provides libraries for every need, including canvas manipulation (critical for mask drawing in inpainting).
- **TypeScript Integration**: Mature TypeScript support enables type-safe development with excellent IDE support.
- **Hooks**: Functional components with hooks provide clean state management without class complexity.
- **Community**: Extensive documentation, tutorials, and community support accelerate development.

3.5.3.4 Supporting Tools

- **Vite**: Fast build tool with hot module replacement, significantly faster than Create React App.
- **Tailwind CSS**: Utility-first CSS framework enabling rapid UI development without writing custom CSS.
- **Canvas API**: Native HTML5 canvas for mask drawing, integrated via React refs.

3.5.4 Deployment Strategy

3.5.4.1 Options Explored

Three deployment environments were evaluated:

- **Google Colab**: Free cloud Jupyter environment with T4 GPU access.
- **NTU HPC Cluster**: University-provided high-performance computing resources.
- **Cloud VM**: Commercial cloud instances (AWS, GCP, Azure) with GPU support.

Table 9: Deployment Environment Comparison

Criteria	**Google Colab**	**NTU HPC**	**Cloud VM**
Cost	Free (T4)	Free (for students)	$1–3/hour (GPU)
GPU Availability	T4 (15GB)	V100/A100	Various
Session Limits	12 hours, idle timeout	Queue-based	Persistent
Public Access	ngrok tunnelling	VPN required	Direct
Setup Complexity	Low (notebook)	Moderate (job scripts)	High (infrastructure)

3.5.4.2 Comparison

3.5.4.3 Decision A hybrid deployment strategy was adopted:

- **Development/Demo**: Google Colab with ngrok tunnelling for rapid prototyping and demonstration. The T4 GPU (15 GB VRAM) supports all models with quantisation.
- **Extended Testing**: NTU HPC for longer-running experiments requiring V100/A100 GPUs without session limits.
- **Frontend Hosting**: Static frontend deployed separately (e.g., Vercel, Netlify) to avoid session timeout issues.

3.5.4.4 Quantisation Strategy To enable deployment on memory-constrained environments like Colab's T4 GPU, DiffusionDesk implements dynamic quantisation:

- **8-bit (LLM.int8)**: Reduces VRAM by $\sim$50% with minimal quality impact. Suitable for SDXL on 8–12 GB GPUs.
- **4-bit (NF4)**: Reduces VRAM by $\sim$75% with some quality trade-off. Enables FLUX.1 Fill on T4 GPU.
- **CPU Offloading**: Sequential layer offloading to CPU RAM when GPU VRAM is exhausted, trading speed for memory.

Users can select quantisation level per model based on their hardware constraints and quality requirements.

4 Software Requirements

This section presents the software requirements for DiffusionDesk, a web-based image editing application powered by diffusion models. The requirements were gathered through a combination of literature review of existing image editing tools, analysis of diffusion model capabilities, and consideration of the target deployment environment (Google Colab with NVIDIA T4 GPU). The requirements are organised into use cases, functional requirements, and non-functional requirements.

4.1 Use Case Diagram

Figure 1 presents the use case diagram for DiffusionDesk, illustrating the interactions between the User actor and the system's core functionalities. The system boundary encompasses ten use cases spanning the three main feature areas: Inpainting, Style Transfer, and Restoration.

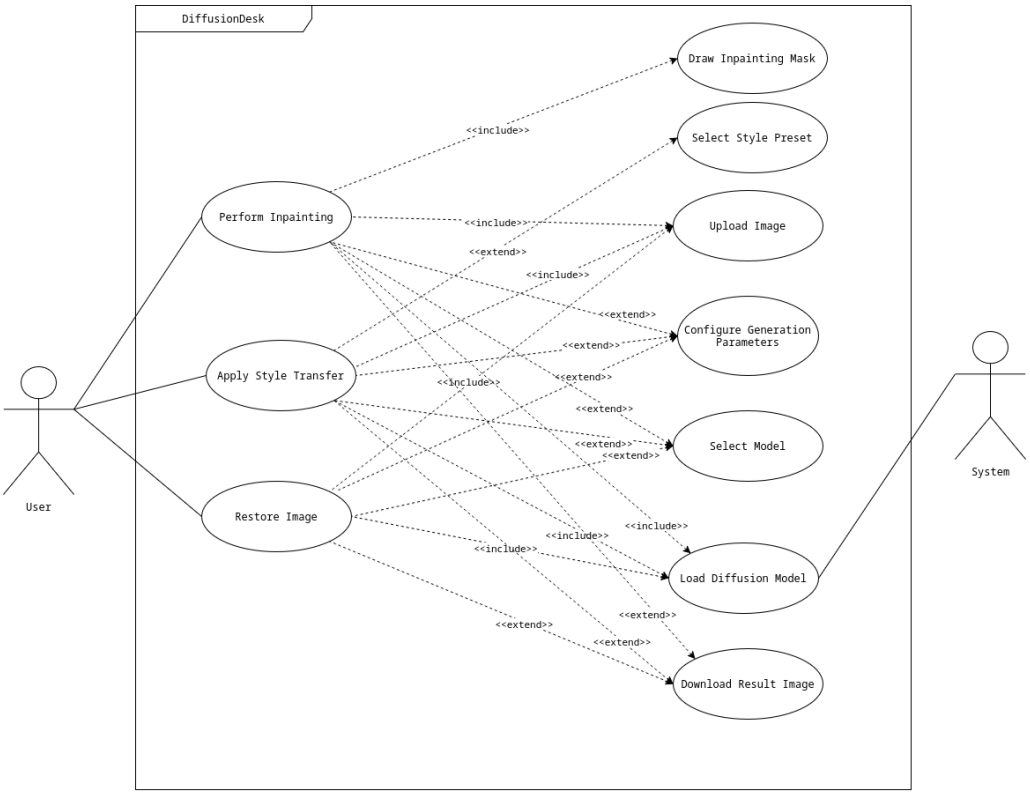

Figure 1: Use Case Diagram for DiffusionDesk

The User actor interacts directly with three primary use cases: UC-2 (Perform Inpainting), UC-4 (Apply Style Transfer), and UC-6 (Restore Image). The remaining use cases are reached indirectly through include and extend relationships. UC-10 (Load Diffusion Model) is associated with the System actor, as it is triggered automatically when a model is needed for inference. Key relationships include:

- **Include relationships**: UC-2, UC-4, and UC-6 all include UC-1 (Upload Image), as an image must be uploaded before any editing operation. UC-2 also includes UC-3 (Draw Inpainting Mask), as a mask is required for inpainting. All editing operations include UC-10 (Load Diffusion Model).
- **Extend relationships**: UC-7 (Select Model) and UC-8 (Configure Generation Parameters) extend the editing use cases, as users may optionally change the model or adjust parameters. UC-5 (Select Style Preset) extends UC-4 (Apply Style Transfer). UC-9 (Download Result Image) extends all three editing use cases, as the user may optionally download the generated result.

4.1.1 Use Case Descriptions

The following tables provide detailed descriptions for each use case identified in the use case diagram. Each table follows a standardised format documenting the use case metadata, flow of events, and related information.

UC-1: Upload Image

Use Case ID	UC-1
Use Case Name	Upload Image
Created By	Lee Yu Quan
Date Created	8 February 2026
Actor	User
Description	The user uploads an image from their local device to the application for subsequent editing operations (inpainting, style transfer, or restoration).
Preconditions	1. The user has navigated to one of the editing tabs (Inpainting, Style Transfer, or Restoration). 2. The user has an image file in a supported format (JPEG, PNG, WebP).
Postconditions	1. The uploaded image is displayed as a preview in the editing canvas. 2. The image is ready for subsequent editing operations.
Priority	High
Frequency of Use	Every editing session

Flow of Events	1. The user clicks the upload area or drag-and-drop zone. 2. The system opens a file selection dialog. 3. The user selects an image file. 4. The system validates the file format and size. 5. The system displays the image preview in the canvas.
Alternative Flows	3a. The user drags and drops an image file onto the upload area instead of using the file dialog.
Exceptions	4a. The file format is not supported: the system displays an error message listing supported formats. 4b. The file exceeds the maximum size limit: the system displays an error with the size constraint.
Includes	–
Special Requirements	Image files must not exceed 10 MB. Supported formats: JPEG, PNG, WebP.
Assumptions	The user has a compatible web browser with JavaScript enabled.
Notes and Issues	Large images may be resized on the backend to fit within model input constraints.

UC-2: Perform Inpainting

Use Case ID	UC-2
Use Case Name	Perform Inpainting
Created By	Lee Yu Quan
Date Created	8 February 2026
Actor	User
Description	The user fills in or replaces a masked region of an uploaded image using a diffusion-based inpainting model. The user provides a text prompt describing the desired content for the masked area.

Preconditions	1. An image has been uploaded (UC-1). 2. A mask has been drawn over the region to inpaint (UC-3). 3. The backend server is running and a GPU is available.
Postconditions	1. The inpainted image is displayed alongside the original for comparison. 2. The result image is available for download (UC-9).
Priority	High
Frequency of Use	Frequent
Flow of Events	1. The user navigates to the Inpainting tab. 2. The user uploads an image (UC-1). 3. The user draws a mask over the region to edit (UC-3). 4. The user enters a text prompt describing the desired content. 5. The user optionally selects a model (UC-7) and configures parameters (UC-8). 6. The user clicks the “Generate” button. 7. The system sends the image, mask, prompt, and parameters to the backend. 8. The system loads the selected model if not already cached (UC-10). 9. The system performs inpainting inference and returns the result. 10. The system displays the inpainted image.
Alternative Flows	4a. The user enters a negative prompt to specify undesired content. 10a. The user is unsatisfied and regenerates with different parameters.
Exceptions	8a. The model fails to load due to insufficient GPU VRAM: the system suggests a lower quantisation level or smaller model. 9a. Inference fails or times out: the system displays an error message and allows the user to retry.
Includes	UC-1 (Upload Image), UC-3 (Draw Inpainting Mask), UC-10 (Load Diffusion Model)

Special Requirements	Requires an active GPU backend. Inference time varies by model (10–120 seconds).
Assumptions	The backend has sufficient VRAM for the selected model and quantisation level.
Notes and Issues	FLUX.1 Fill models require NF4 quantisation to run on T4 GPUs.

UC-3: Draw Inpainting Mask

Use Case ID	UC-3
Use Case Name	Draw Inpainting Mask
Created By	Lee Yu Quan
Date Created	8 February 2026
Actor	User
Description	The user draws a binary mask on the uploaded image to indicate the region that should be inpainted. White pixels in the mask denote the area to be filled.
Preconditions	1. An image has been uploaded and displayed in the Inpainting tab.
Postconditions	1. A mask overlay is visible on the image canvas. 2. The mask data is ready to be sent with the inpainting request.
Priority	High
Frequency of Use	Every inpainting operation
Flow of Events	1. The user selects the brush tool on the canvas. 2. The user adjusts the brush size using the slider control. 3. The user draws over the region to be inpainted by clicking and dragging on the canvas. 4. The system renders the mask overlay in a semi-transparent colour. 5. The user reviews the mask coverage.

Alternative Flows	3a. The user uses the eraser tool to remove parts of the mask. 3b. The user clicks “Clear Mask” to reset the entire mask.
Exceptions	3a. No image is loaded: the canvas tools are disabled.
Includes	–
Special Requirements	The canvas must support touch input for tablet/touchscreen devices.
Assumptions	The user can use a mouse or touchscreen to draw on the canvas.
Notes and Issues	The mask is internally represented as a black-and-white image matching the uploaded image dimensions.

UC-4: Apply Style Transfer

Use Case ID	UC-4
Use Case Name	Apply Style Transfer
Created By	Lee Yu Quan
Date Created	8 February 2026
Actor	User
Description	The user applies an artistic style (e.g., anime, oil painting, watercolour, pixel art) to an uploaded image using a diffusion-based image-to-image pipeline.
Preconditions	1. An image has been uploaded (UC-1). 2. The backend server is running and a GPU is available.
Postconditions	1. The stylised image is displayed alongside the original. 2. The result image is available for download (UC-9).
Priority	High
Frequency of Use	Frequent

Flow of Events	1. The user navigates to the Style Transfer tab. 2. The user uploads an image (UC-1). 3. The user selects a style preset (UC-5) or enters a custom style prompt. 4. The user optionally selects a model (UC-7) and adjusts the strength parameter to control style intensity. 5. The user optionally configures additional parameters (UC-8). 6. The user clicks the "Generate" button. 7. The system sends the image, prompt, and parameters to the backend. 8. The system loads the model if not already cached (UC-10). 9. The system performs image-to-image inference and returns the result. 10. The system displays the stylised image.
Alternative Flows	3a. The user enters a custom prompt instead of selecting a preset. 10a. The user adjusts the strength and regenerates to fine-tune the result.
Exceptions	8a. Model fails to load: the system suggests a lower quantisation level. 9a. Inference fails: the system displays an error message.
Includes	UC-1 (Upload Image), UC-10 (Load Diffusion Model)
Special Requirements	Requires an active GPU backend.
Assumptions	The backend has sufficient VRAM for the SDXL img2img model.
Notes and Issues	Higher strength values produce more stylised results but may lose original content details. A strength of 0.5–0.7 is recommended for most styles.

UC-5: Select Style Preset

Use Case ID	UC-5
Use Case Name	Select Style Preset

Created By	Lee Yu Quan
Date Created	8 February 2026
Actor	User
Description	The user selects a predefined artistic style from a list of presets. Each preset maps to a curated prompt optimised for that style.
Preconditions	1. The user is on the Style Transfer tab.
Postconditions	1. The selected style preset is applied to the prompt field. 2. The style is ready to be used in the next generation.
Priority	Medium
Frequency of Use	Frequent
Flow of Events	1. The user views the list of available style presets (e.g., Anime, Oil Painting, Watercolour, Pixel Art, Cinematic, Pencil Sketch). 2. The user clicks on a style preset. 3. The system populates the prompt field with the preset's curated prompt.
Alternative Flows	3a. The user modifies the auto-populated prompt to customise the style further.
Exceptions	–
Includes	–
Special Requirements	–
Assumptions	The list of presets is fetched from the backend API.
Notes and Issues	Presets can be extended by adding new entries to the backend configuration.

UC-6: Restore Image

Use Case ID	UC-6
Use Case Name	Restore Image
Created By	Lee Yu Quan

Date Created	8 February 2026
Actor	User
Description	The user restores or enhances a degraded image using one of the available restoration models: CodeFormer and GFP-GAN for face restoration, or Real-ESRGAN for general image upscaling and enhancement.
Preconditions	1. An image has been uploaded (UC-1). 2. The backend server is running and a GPU is available.
Postconditions	1. The restored image is displayed alongside the original. 2. The result image is available for download (UC-9).
Priority	High
Frequency of Use	Frequent
Flow of Events	1. The user navigates to the Restoration tab. 2. The user uploads an image (UC-1). 3. The user optionally selects a restoration model (UC-7) (e.g., CodeFormer, GFPGAN, or Real-ESRGAN). 4. The user optionally adjusts model-specific parameters (UC-8) (e.g., fidelity weight for CodeFormer, upscale factor for Real-ESRGAN). 5. The user clicks the “Restore” button. 6. The system sends the image and parameters to the backend. 7. The system loads the selected model (UC-10). 8. The system performs restoration inference and returns the result. 9. The system displays the restored image.
Alternative Flows	3a. The user tries a different restoration model on the same image.
Exceptions	3a. Face restoration models (CodeFormer, GFPGAN) produce suboptimal results on images without detectable faces: the system returns the image with minimal changes. 8a. Inference fails: the system displays an error message.
Includes	UC-1 (Upload Image), UC-10 (Load Diffusion Model)

Special Requirements	Requires an active GPU backend. Face restoration models work best on images containing human faces.
Assumptions	The uploaded image contains content suitable for the selected restoration model.
Notes and Issues	Real-ESRGAN can upscale images by $2\times$ or $4\times$, which increases output resolution and file size.

UC-7: Select Model

Use Case ID	UC-7
Use Case Name	Select Model
Created By	Lee Yu Quan
Date Created	8 February 2026
Actor	User
Description	The user selects a specific AI model from the list of available models for the current editing operation. Different models offer varying trade-offs between quality, speed, and VRAM requirements.
Preconditions	1. The user is on an editing tab (Inpainting, Style Transfer, or Restoration).
Postconditions	1. The selected model is stored and will be used for the next generation request.
Priority	Medium
Frequency of Use	Occasional
Flow of Events	1. The user views the model selection dropdown. 2. The system displays available models with their names. 3. The user selects a model from the dropdown. 4. The system updates the selected model for subsequent requests.
Alternative Flows	1a. The user keeps the default model selection.
Exceptions	–

Includes	–
Special Requirements	The model list should be fetched dynamically from the backend API.
Assumptions	The backend returns a list of models appropriate for the current task.
Notes and Issues	Available inpainting models include SD Inpainting, SDXL Inpainting, Kandinsky, and FLUX.1 Fill. Style transfer uses SDXL img2img. Restoration models include CodeFormer, GFPGAN, and Real-ESRGAN.

UC-8: Configure Generation Parameters

Use Case ID	UC-8
Use Case Name	Configure Generation Parameters
Created By	Lee Yu Quan
Date Created	8 February 2026
Actor	User
Description	The user adjusts generation parameters that control the behaviour and quality of the AI inference, such as guidance scale, number of inference steps, strength, and random seed.
Preconditions	1. The user is on an editing tab with parameter controls visible.
Postconditions	1. The configured parameters are stored and will be sent with the next generation request.
Priority	Medium
Frequency of Use	Occasional

Flow of Events	1. The user expands the advanced parameters section. 2. The user adjusts one or more parameters using sliders or input fields: • Guidance scale (1.0–20.0) • Number of inference steps (1–100) • Strength (0.0–1.0, for img2img/style transfer) • Random seed (integer, or -1 for random) • Quantisation level (none, 8-bit, 4-bit) 3. The system validates the parameter values against allowed ranges. 4. The system updates the stored parameters.
Alternative Flows	1a. The user does not expand the advanced section and uses default values.
Exceptions	3a. A parameter value is out of range: the system clamps it to the nearest valid value.
Includes	–
Special Requirements	Parameter controls must provide sensible default values.
Assumptions	The user understands the effect of advanced parameters or is satisfied with defaults.
Notes and Issues	Increasing inference steps improves quality but increases generation time. Guidance scale controls prompt adherence.

UC-9: Download Result Image

Use Case ID	UC-9
Use Case Name	Download Result Image
Created By	Lee Yu Quan
Date Created	8 February 2026
Actor	User
Description	The user downloads the generated or restored image to their local device.

Preconditions	1. A result image has been generated by one of the editing operations.
Postconditions	1. The result image is saved to the user's local device.
Priority	Medium
Frequency of Use	After each successful generation
Flow of Events	1. The user clicks the "Download" button below the result image. 2. The system triggers a file download in the browser. 3. The image is saved to the user's default download directory.
Alternative Flows	1a. The user right-clicks the image and selects "Save Image As" from the browser context menu.
Exceptions	–
Includes	–
Special Requirements	The downloaded image should be in PNG format to preserve quality.
Assumptions	The user's browser supports programmatic file downloads.
Notes and Issues	The filename includes a timestamp and operation type for easy identification.

UC-10: Load Diffusion Model

Use Case ID	UC-10
Use Case Name	Load Diffusion Model
Created By	Lee Yu Quan
Date Created	8 February 2026
Actor	System
Description	The system loads the requested diffusion model into GPU memory, applying quantisation if specified. Previously loaded models are cached to avoid redundant loading.

Preconditions	1. An editing operation has been requested (UC-2, UC-4, or UC-6). 2. The requested model is not already cached in GPU memory.
Postconditions	1. The model is loaded into GPU memory and ready for inference. 2. The model is cached for subsequent requests.
Priority	High
Frequency of Use	On first use of each model per session
Flow of Events	1. The system receives a generation request specifying a model. 2. The system checks if the model is already cached in memory. 3. If not cached, the system downloads the model weights from Hugging Face Hub (or loads from local cache). 4. The system applies the requested quantisation level (none, 8-bit, or 4-bit). 5. The system loads the model onto the GPU. 6. The system caches the loaded model for future requests. 7. The system returns control to the calling use case for inference.
Alternative Flows	2a. The model is already cached: skip to step 7. 4a. If the model requires CPU offloading, the system enables sequential offloading.
Exceptions	3a. Model weights are unavailable (network error or missing from Hub): the system returns an error to the user. 5a. Insufficient GPU VRAM: the system clears cached models and retries, or returns an error suggesting a lower quantisation level.
Includes	–
Special Requirements	Model loading can take 30–120 seconds depending on model size and network speed. A progress indicator should be shown to the user.

Assumptions	The backend has internet access to download model weights on first use.
Notes and Issues	Switching between large models (e.g., SDXL to FLUX.1) may require clearing the previous model from VRAM first.

4.2 Functional and Non-Functional Requirements

The functional and non-functional requirements for DiffusionDesk were derived from the use case analysis above, supplemented by best practices in web application development and the specific constraints of deploying diffusion models on consumer-grade GPUs. Functional requirements define the system's behaviour, while non-functional requirements specify quality attributes and constraints.

4.2.1 Functional Requirements

The functional requirements are organised by use case, with each requirement identified by a unique code in the format **FR-XX**.

Upload Image

FR-1. The system shall accept image uploads in JPEG, PNG, and WebP formats.

FR-2. The system shall reject image files exceeding 10 MB and display an appropriate error message.

FR-3. The system shall display a preview of the uploaded image in the editing canvas.

FR-4. The system shall support drag-and-drop image upload in addition to the file selection dialog.

FR-5. The system shall resize images exceeding the model's maximum input resolution while preserving the aspect ratio.

Perform Inpainting

FR-6. The system shall provide a selection of inpainting models (SD Inpainting, SDXL Inpainting, Kandinsky Inpainting, FLUX.1 Fill).

FR-7. The system shall accept a text prompt describing the desired content for the masked region.

FR-8. The system shall accept an optional negative prompt to specify undesired content.

FR-9. The system shall send the image, mask, prompt, negative prompt, and generation parameters to the backend API.

FR-10. The system shall display the inpainted result image alongside the original for visual comparison.

Draw Inpainting Mask

FR-11. The system shall provide a canvas-based brush tool for drawing masks over the uploaded image.

FR-12. The system shall allow the user to adjust the brush size via a slider control.

FR-13. The system shall render the mask as a semi-transparent overlay on the image.

FR-14. The system shall provide a "Clear Mask" button to reset the entire mask.

FR-15. The system shall generate a binary mask image (black background, white mask) matching the uploaded image dimensions.

Apply Style Transfer

FR-16. The system shall support style transfer via diffusion-based image-to-image generation.

FR-17. The system shall provide predefined style presets (e.g., Anime, Oil Painting, Watercolour, Pixel Art, Cinematic, Pencil Sketch).

FR-18. The system shall allow the user to enter a custom style prompt.

FR-19. The system shall provide a strength parameter (0.0–1.0) to control the intensity of the style application.

FR-20. The system shall display the stylised result image alongside the original for comparison.

Restore Image

FR-21. The system shall support face restoration using CodeFormer and GFPGAN models.

FR-22. The system shall support general image upscaling and enhancement using Real-ESRGAN.

FR-23. The system shall provide model-specific parameters (e.g., fidelity weight for CodeFormer, upscale factor for Real-ESRGAN).

FR-24. The system shall display the restored result image alongside the original for comparison.

Select Model

FR-25. The system shall fetch the list of available models from the backend API dynamically.

FR-26. The system shall display the available models in a dropdown selection menu.

FR-27. The system shall set a sensible default model for each editing tab.

Configure Generation Parameters

FR-28. The system shall provide controls for adjusting guidance scale (1.0–20.0).

FR-29. The system shall provide controls for adjusting the number of inference steps (1–100).

FR-30. The system shall provide controls for adjusting strength (0.0–1.0) for image-to-image operations.

FR-31. The system shall provide a seed input field for reproducible generation (-1 for random).

FR-32. The system shall provide quantisation level selection (none, 8-bit, 4-bit) where applicable.

FR-33. The system shall validate parameter values and clamp them to valid ranges.

FR-34. The system shall provide sensible default values for all parameters.

Download Result Image

FR-35. The system shall provide a download button for each generated result image.

FR-36. The system shall download the image in PNG format with a descriptive filename.

Load Diffusion Model

FR-37. The system shall load diffusion models on demand when a generation request is received.

FR-38. The system shall cache loaded models in GPU memory to avoid redundant loading.

FR-39. The system shall apply quantisation (8-bit or 4-bit) when specified by the user.

FR-40. The system shall clear previously cached models when GPU VRAM is insufficient for a new model.

FR-41. The system shall display a loading indicator while a model is being loaded.

4.2.2 Non-Functional Requirements

The non-functional requirements define the quality attributes and constraints of DiffusionDesk. Each requirement is identified by a unique code in the format **NFR-XX**.

Usability

NFR-1. The user interface shall be intuitive and require no prior experience with diffusion models to use basic features.

NFR-2. The system shall provide clear labels, tooltips, and instructions for all controls.

NFR-3. The system shall maintain a consistent visual design across all tabs using Tailwind CSS.

NFR-4. The system shall provide real-time feedback during image generation, including progress indicators and status messages.

NFR-5. The system shall organise features into clearly labelled tabs (Inpainting, Style Transfer, Restoration) for easy navigation.

NFR-6. The system shall display error messages in a user-friendly format, avoiding raw technical error traces.

Performance

NFR-7. The frontend shall load within 3 seconds on a standard broadband connection.

NFR-8. Image upload and preview rendering shall complete within 2 seconds for files under 5 MB.

NFR-9. Inpainting and style transfer inference shall complete within 120 seconds on a T4 GPU with 4-bit quantisation.

NFR-10. Image restoration shall complete within 30 seconds on a T4 GPU.

NFR-11. The system shall display generation progress to the user so that long-running operations do not appear unresponsive.

Reliability

NFR-12. The system shall handle backend errors gracefully and display meaningful error messages to the user.

NFR-13. The system shall recover from GPU out-of-memory errors by suggesting alternative models or quantisation levels.

NFR-14. The frontend shall remain functional even if the backend is temporarily unavailable, allowing image upload and parameter configuration.

NFR-15. The system shall validate all user inputs on both the frontend and backend to prevent invalid requests.

Maintainability

NFR-16. The system shall follow a modular architecture with clear separation between frontend, backend API, and ML inference services.

NFR-17. The backend shall use a router-service pattern with low coupling between API endpoints and model inference logic.

NFR-18. The frontend shall use reusable React components with well-defined props interfaces.

NFR-19. New models shall be addable to the system by extending the backend service configuration without modifying existing code.

Compatibility

NFR-20. The frontend shall be compatible with the latest versions of Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari.

NFR-21. The frontend shall be responsive and functional on screen widths from 1024 pixels and above.

NFR-22. The backend shall run on NVIDIA GPUs with CUDA support (compute capability 7.0 or higher).

NFR-23. The system shall support deployment on Google Colab with T4 GPU using ngrok for backend exposure.

5 Planning and Design

This section presents the planning and design decisions made during the development of DiffusionDesk. It covers the software development methodology adopted, the system architecture, and the user interface wireframe designs that guided the implementation.

5.1 Project Development Methodology

The software development methodology adopted for DiffusionDesk is the **Iterative and Incremental Development** methodology. This approach was chosen because the project involves integrating multiple AI models with varying hardware requirements, where each feature (inpainting, style transfer, restoration) can be developed and tested as an independent increment. The iterative nature allows for continuous refinement based on testing feedback, which is particularly important when working with diffusion models where output quality depends heavily on parameter tuning and model selection.

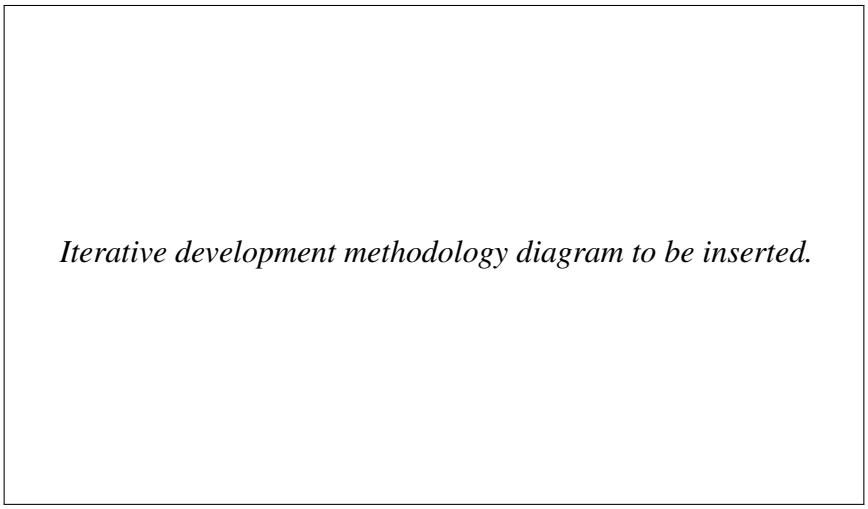

Figure 2: Iterative and Incremental Development Methodology

The project was divided into four main iterations, each building upon the previous:

1. **Iteration 1 — Project Setup and Backend Foundation**: Established the FastAPI backend structure with the router-service architecture pattern. Set up the project repository, defined API schemas using Pydantic, and implemented the health check endpoint. The frontend scaffolding was created using React, TypeScript, and Vite with Tailwind CSS for styling.

2. **Iteration 2 — Inpainting Feature**: Implemented the inpainting service with support for multiple diffusion models (SD Inpainting, SDXL Inpainting, Kandinsky, FLUX.1 Fill). Developed the frontend `InpaintingTab` component with the interactive `MaskCanvas` for drawing masks. Integrated quantisation support (8-bit and 4-bit) using bitsandbytes to enable deployment on memory-constrained GPUs.

3. **Iteration 3 — Style Transfer and Restoration Features**: Added the style transfer service using SDXL img2img pipelines with predefined style presets. Implemented the restoration service integrating CodeFormer, GFPGAN, and Real-ESRGAN models. Developed the corresponding frontend tabs (`StyleTransferTab` and `RestorationTab`).

4. **Iteration 4 — Integration Testing and Deployment**: Configured Google Colab deployment with ngrok for backend exposure. Tested all three features end-to-end on T4 GPU hardware. Refined parameter defaults and error handling based on testing results.

At the end of each iteration, the working software was reviewed and evaluated against the project requirements. Feedback from testing on actual GPU hardware informed the subsequent iteration, particularly regarding VRAM constraints and quantisation strategies. This iterative approach proved well-suited for the project, as requirements around model support and parameter ranges evolved as new models were tested and evaluated.

5.2 System Architecture

DiffusionDesk adopts a **client-server architecture** with a clear separation between the frontend application and the backend API server. The frontend is a single-page application (SPA) built with React that communicates with the backend via RESTful HTTP requests. The backend is a FastAPI server that handles API routing, request validation, and delegates ML inference to dedicated service modules. Figure 3 illustrates the overall system architecture.

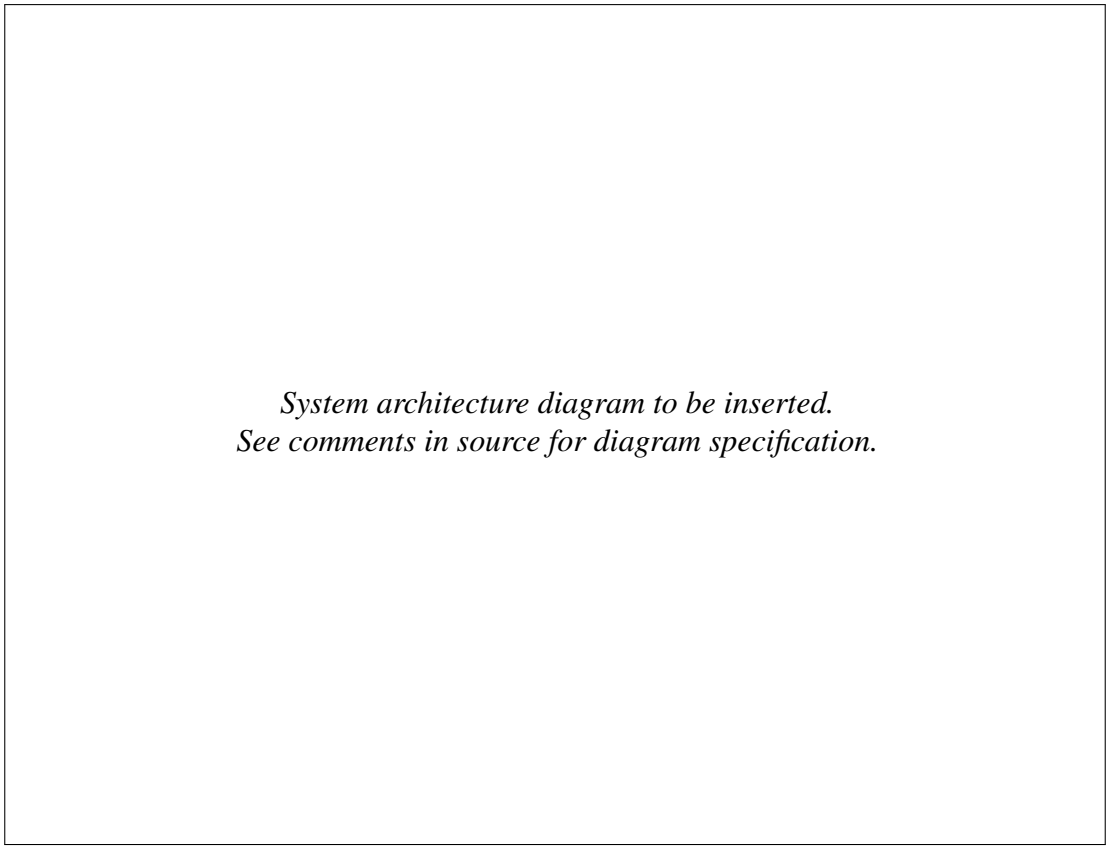

Figure 3: System Architecture of DiffusionDesk

The architecture consists of two main components: the **Frontend Application** and the **Backend API Server**, connected via HTTP/JSON communication. This separation enables independent deployment — the frontend can be hosted on static hosting platforms such as Vercel or Netlify, while the backend runs on a GPU-equipped machine such as Google Colab.

5.2.1 Frontend Architecture

The frontend is a React 19 single-page application built with TypeScript and bundled using Vite. Tailwind CSS provides utility-first styling with a dark theme (slate-900 background with indigo accents). The application is organised into the following layers:

- **App Component** (`App.tsx`): The root component manages tab navigation through an `activeTab` state variable. Four tabs are available: Home, Inpainting, Style Transfer, and Restoration. Each tab renders its corresponding component.
- **Tab Components** (`components/`): Each editing feature is encapsulated in its own tab component — `InpaintingTab.tsx`, `StyleTransferTab.tsx`, and `RestorationTab.tsx`. Each tab manages its own local state for image data, generation parameters, and UI state (loading, errors, results). This design ensures that tabs are independent and do not share mutable state.

- **Shared Components**: Reusable components include `ImageUpload` (drag-and-drop file upload with preview) and `MaskCanvas` (HTML5 Canvas-based interactive drawing tool for creating inpainting masks). These are used across multiple tabs to maintain consistency.
- **API Client** (`api/`): The `imageApi.ts` module provides typed API functions (`inpaintImage()`, `styleTransfer()`, `restoreImage()`) that handle base64 encoding, request construction, and response parsing. The `config.ts` module reads the API base URL from the `VITE_API_URL` environment variable, enabling seamless switching between local and cloud-deployed backends.

5.2.2 Backend Architecture

The backend follows a **three-layer router-service-schema architecture** built on FastAPI. This pattern separates concerns cleanly and mirrors the structure used in production web services:

- **Router Layer** (`routers/`): Three router modules — `inpainting.py`, `style_transfer.py`, and `restoration.py` — define the API endpoints. Each router receives HTTP requests, extracts validated parameters from Pydantic schemas, delegates processing to the corresponding service, and returns a standardised `ImageResponse`. Routers are registered in `main.py` with URL prefixes: `/api/inpainting/`, `/api/style/`, and `/api/restoration/`.
- **Schema Layer** (`schemas/image.py`): Pydantic models define the API contracts. `InpaintingRequest`, `StyleTransferRequest`, and `RestorationRequest` validate incoming data with type checking and range constraints (e.g., `guidance_scale` between 1.0 and 20.0, `num_inference_steps` between 10 and 100). The shared `ImageResponse` model provides a consistent response format with fields for `success`, `image` (base64), `error`, `model_used`, and `processing_time`.
- **Service Layer** (`services/`): The `DiffusionService` class manages diffusion model pipelines for inpainting and style transfer, while the `RestorationService` handles CodeFormer, GFPGAN, and Real-ESRGAN models. Services are instantiated as **singletons** using factory functions (`get_diffusion_service()` and `get_restoration_service()`), ensuring that loaded models are cached in GPU memory across multiple requests. This avoids redundant model loading and reduces response latency for subsequent requests.

5.2.3 Model Management Strategy

A key architectural decision is the **lazy loading with caching** strategy for ML models. Models are not loaded at server startup; instead, they are loaded on demand when the first request for a

given model is received. Once loaded, the model pipeline is cached in a dictionary keyed by model identifier. This approach offers several benefits:

- **Memory efficiency**: Only models that are actively used consume GPU VRAM.
- **Fast subsequent requests**: Cached models skip the loading phase (which can take 30–120 seconds) on repeat use.
- **Flexible model switching**: Users can select different models per request without preloading all options.
- **Quantisation on demand**: Models can be loaded with different precision levels (FP16, 8-bit, 4-bit NF4) based on the user's hardware constraints.

When GPU VRAM is insufficient for a new model, the system clears previously cached models before attempting to load the requested model.

5.2.4 Communication Protocol

The frontend and backend communicate via **RESTful JSON APIs**. Images are transmitted as base64-encoded strings within JSON payloads, which simplifies the API design by avoiding multipart form data. The data flow for a typical editing operation proceeds as follows:

1. The user uploads an image file in the browser. The frontend converts it to a base64 data URL using the `FileReader` API.
2. The user configures parameters (prompt, model, generation settings) through the UI controls.
3. The frontend constructs a JSON payload containing the base64 image, parameters, and (for inpainting) the mask, and sends it as a POST request to the corresponding API endpoint.
4. The backend validates the request using Pydantic schemas, decodes the base64 images to PIL Image objects, and invokes the appropriate service method.
5. The service loads the model if needed, performs GPU inference, and encodes the result image back to base64.
6. The backend returns an `ImageResponse` JSON object containing the result image, model used, and processing time.
7. The frontend converts the base64 response to a data URL and displays the result alongside the original image for comparison.

5.2.5 Deployment Architecture

DiffusionDesk is designed for flexible deployment across different environments:

- **Local development**: The frontend runs on `localhost:5173` (Vite dev server) and the backend on `localhost:8000` (Uvicorn). CORS middleware configured with `allow_origins=["*"]` permits cross-origin requests between the two ports.
- **Google Colab deployment**: The backend runs inside a Colab notebook with GPU acceleration. Ngrok exposes the FastAPI server via a public HTTPS URL. The frontend is configured to point to the ngrok URL through the `VITE_API_URL` environment variable and can be hosted on a static platform or run locally.
- **Production deployment**: The frontend is built to static files using `npm run build` and deployed to Vercel or Netlify. The backend is deployed on a GPU-equipped server (e.g., NTU HPC cluster with V100/A100 GPUs).

5.3 User Interface Wireframe

This section presents the wireframe designs for the key pages of the DiffusionDesk web application. These wireframes provide a visual representation of the interface's layout and functionality, serving as a blueprint for the frontend implementation. The wireframes were designed to prioritise usability, with clear separation of controls, input areas, and result displays.

Home Page

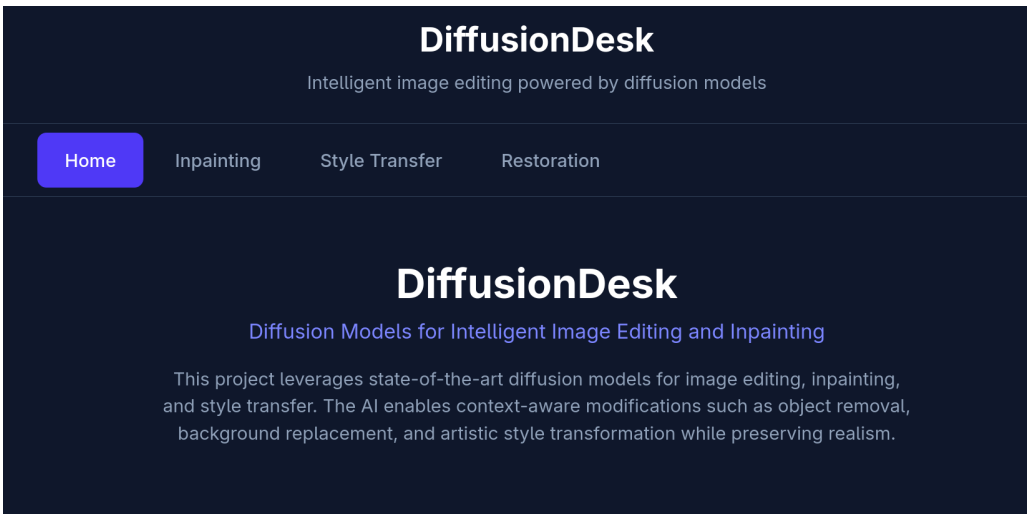

Figure 4: Home Page (Wireframe)

The Home page serves as the landing page for DiffusionDesk. It provides an overview of the application's three core features — Inpainting, Style Transfer, and Restoration — presented as feature cards with brief descriptions. The top navigation bar displays the application name

and tab buttons for navigating to each feature. This page is designed to orient new users and provide quick access to any feature.

Inpainting Page

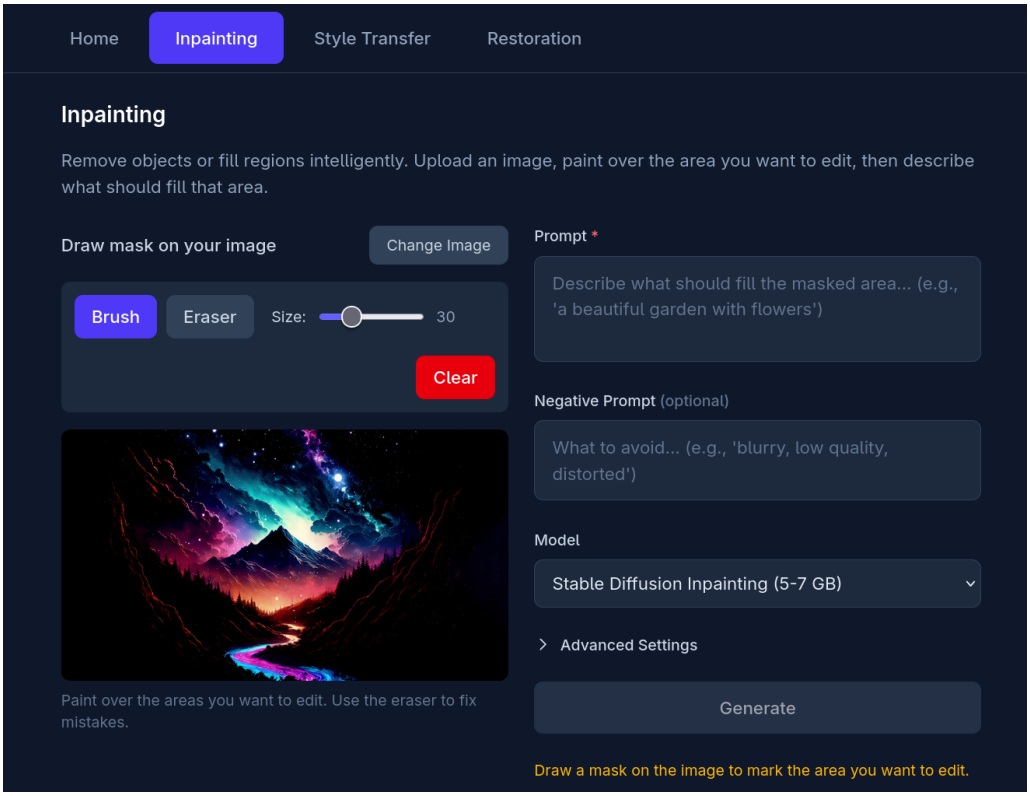

Figure 5: Inpainting Page (Wireframe)

The Inpainting page is the most complex interface in DiffusionDesk. The left side features the image canvas with a mask overlay, where users draw over the region to be inpainted using a configurable brush tool. A brush size slider and "Clear Mask" button are positioned below the canvas. The right side contains the control panel: model selection dropdown, text prompt input, negative prompt input, and a collapsible "Advanced Settings" section with sliders for guidance scale, inference steps, strength, seed input, and quantisation level selection. The "Generate" button triggers the inpainting operation, and the result is displayed below in a side-by-side comparison with the original image. A "Download" button allows saving the result.

The "Generate" button is disabled until three conditions are met: an image is uploaded, a mask is drawn, and a prompt is entered. Inline hints inform the user of any missing requirements.

Style Transfer Page

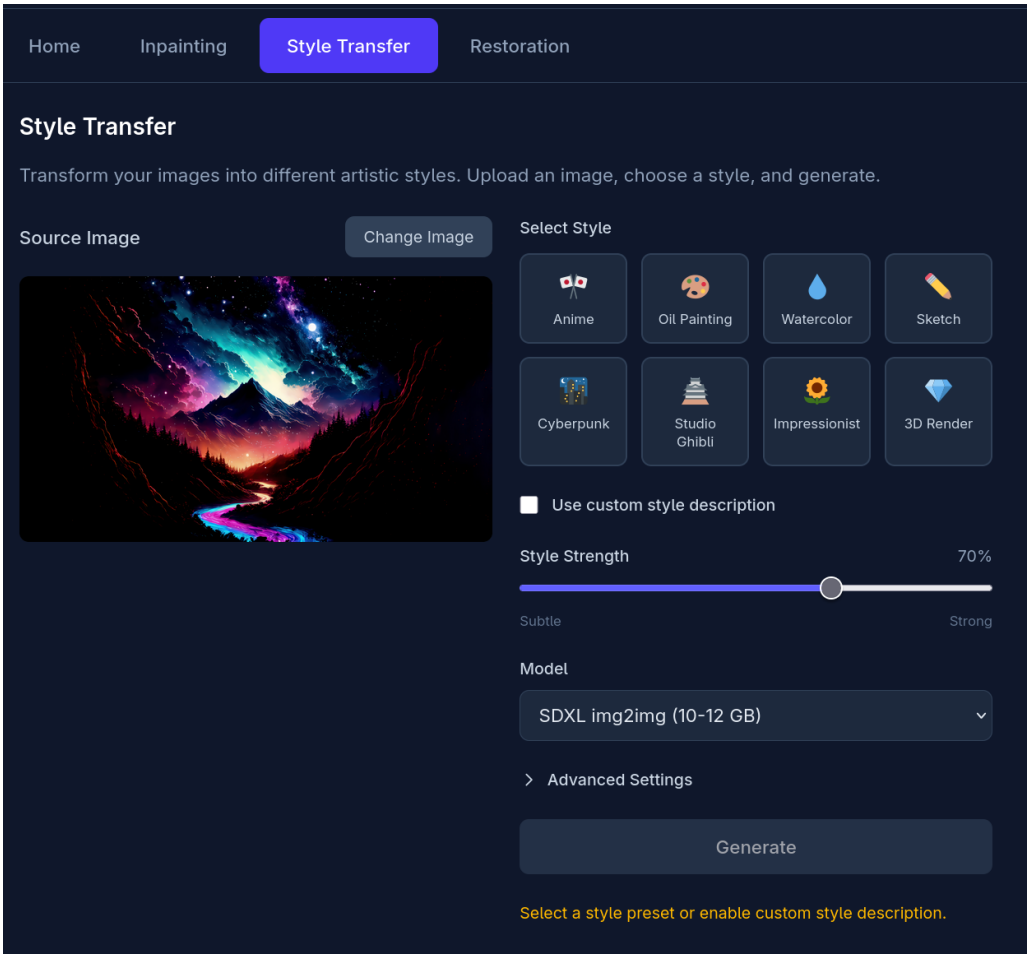

Figure 6: Style Transfer Page (Wireframe)

The Style Transfer page provides a streamlined interface for applying artistic styles to images. The left side displays the uploaded image preview with an upload control. The right side features a grid of style preset buttons (Anime, Oil Painting, Watercolour, Pixel Art, Cinematic, Pencil Sketch, Cyberpunk, Pop Art), each with a visual icon and label. Users can alternatively enable a custom prompt checkbox to enter a free-text style description. A prominent strength slider (0.0–1.0, default 0.7) controls the intensity of the style application — lower values preserve more of the original content, while higher values produce more stylised results. The model selection dropdown and collapsible advanced settings (guidance scale, inference steps, seed) are also available. Results are displayed in the same side-by-side comparison format as the Inpainting page.

Restoration Page

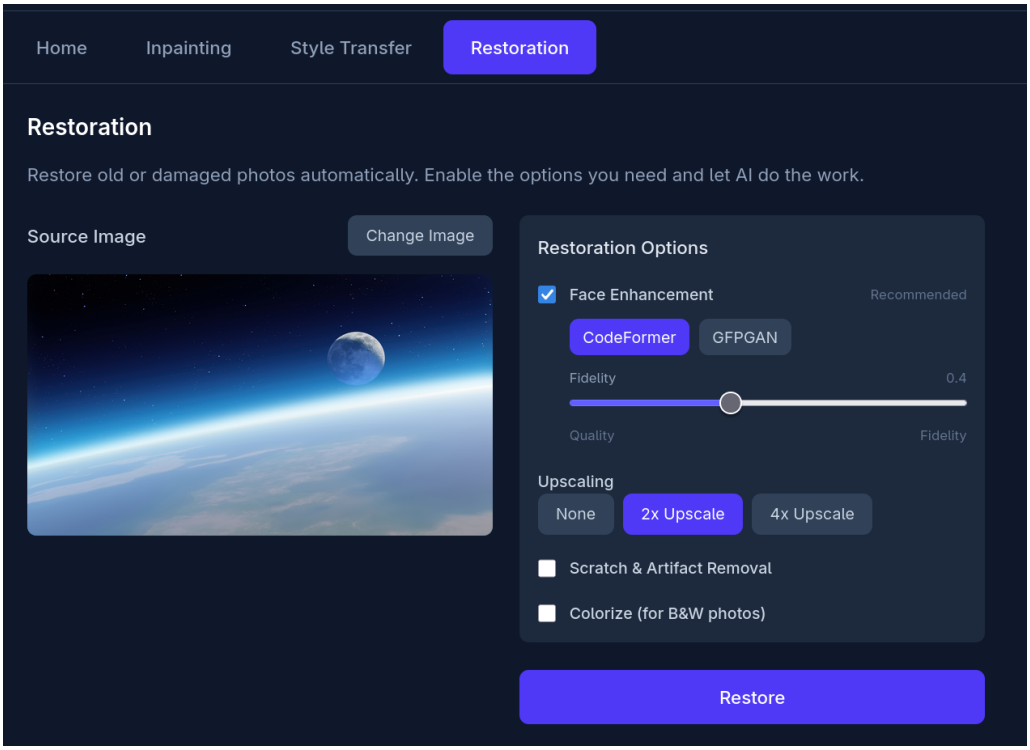

Figure 7: Restoration Page (Wireframe)

The Restoration page offers a checkbox-based interface for selecting restoration operations. The primary options include: Face Enhancement with a model selector (CodeFormer or GFPGAN) and a fidelity slider (for CodeFormer, controlling the balance between quality enhancement and original fidelity); Upscaling with radio buttons for scale factor (None, $2\times$, $4\times$) using Real-ESRGAN; and optional Scratch Removal and Colourisation toggles. The "Restore" button is disabled unless at least one restoration option is enabled. This page does not require a text prompt, making it simpler than the Inpainting and Style Transfer pages. Results are displayed in the same side-by-side comparison format with a download option.

6 Implementation

This section presents the implementation of DiffusionDesk, covering the backend services that perform AI-powered image processing (Section 6.1) and the frontend application that provides the user interface (Section 6.2).

6.1 Backend Development

The backend is built with FastAPI, a modern Python web framework chosen for its automatic request validation via Pydantic, built-in OpenAPI documentation, and native `async` support. The backend is designed to run on GPU-equipped machines (Google Colab, cloud instances, or local workstations).

6.1.1 Project Structure

The backend follows a layered architecture that separates routing, validation, and inference concerns. API requests flow through three layers: **routers** handle HTTP endpoints and delegate to **services**, which contain the ML inference logic; **schemas** define Pydantic models for request validation and response serialisation. Each service is implemented as a singleton to ensure that expensive GPU models are loaded only once and reused across requests.

```
backend/
  app/
    main.py                  # FastAPI app, CORS, router mounting
    routers/
      inpainting.py          # POST /api/inpainting/
      style_transfer.py      # POST /api/style/
      restoration.py         # POST /api/restoration/
    services/
      diffusion.py           # DiffusionService (inpainting + style)
      restoration.py         # RestorationService (faces + upscale)
    schemas/
      image.py               # Pydantic request/response models
  requirements.txt
```

Listing 1: Backend directory structure

Listing 2 shows the application entry point. The `lifespan` context manager handles startup and shutdown events. CORS middleware is configured to allow all origins, which is necessary when the frontend is served from a different domain (e.g., Vercel) while the backend runs on a Colab instance exposed via ngrok. The three feature routers are mounted under `/api/` prefixes, and a health-check endpoint reports GPU availability and loaded pipelines.

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from contextlib import asynccontextmanager
from app.routers import inpainting, style_transfer, restoration

@asynccontextmanager
async def lifespan(app: FastAPI):
    print("Starting up Diffusion Image Editor API...")
    yield
    print("Shutting down...")

app = FastAPI(
    title="Diffusion Image Editor API",
    description="API for intelligent image editing "
                "using diffusion models",
    version="0.1.0",
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(inpainting.router,
    prefix="/api/inpainting", tags=["Inpainting"])
app.include_router(style_transfer.router,
    prefix="/api/style", tags=["Style Transfer"])
app.include_router(restoration.router,
    prefix="/api/restoration", tags=["Restoration"])

@app.get("/api/health")
async def health_check():
    from app.services.diffusion import get_diffusion_service
    service = get_diffusion_service()
    gpu_info = service.get_gpu_info()
    return {
        "status": "healthy",
```

```
        "gpu": gpu_info,
        "loaded_pipelines": list(
            service._pipelines.keys()),
    }
```

Listing 2: Application entry point (`main.py`)

6.1.2 API Design

The API uses JSON request bodies with base64-encoded images rather than multipart form uploads. This design simplifies the frontend integration—the React client can construct the entire request as a single `JSON.stringify()` call—and allows Pydantic to validate all fields (including numeric constraints) in one pass. Every endpoint returns a uniform `ImageResponse` containing a success flag, the result image (base64), the model used, and the processing time.

Listing 3 shows the response model and the inpainting request schema. Pydantic `Field` constraints enforce value ranges at the API boundary: `guidance_scale` is clamped between 1.0 and 20.0, `num_inference_steps` between 10 and 100, and `seed` between 0 and $2^{31}-1$. The `InpaintingModel` enum restricts model selection to known-valid keys, preventing typos from reaching the service layer.

```
class ImageResponse(BaseModel):
    success: bool
    image: Optional[str] = Field(
        None, description="Base64␣encoded␣result␣image")
    error: Optional[str] = None
    model_used: Optional[str] = None
    processing_time: Optional[float] = None

class InpaintingModel(str, Enum):
    SD_INPAINTING = "sd-inpainting"
    KANDINSKY = "kandinsky-inpainting"
    SDXL_INPAINTING = "sdxl-inpainting"
    SDXL_INPAINTING_8BIT = "sdxl-inpainting-8bit"
    SDXL_INPAINTING_4BIT = "sdxl-inpainting-4bit"
    FLUX_FILL = "flux-fill"
    FLUX_FILL_8BIT = "flux-fill-8bit"
    FLUX_FILL_4BIT = "flux-fill-4bit"
    FLUX_FILL_NF4 = "flux-fill-nf4"

class InpaintingRequest(BaseModel):
    image: str = Field(...,
        description="Base64␣encoded␣input␣image")
```

```
    mask: str = Field(...,
        description="Base64 mask (white=inpaint)")
    prompt: str = Field(...,
        description="What to generate in masked area")
    negative_prompt: str = Field(
        default="blurry, low quality, distorted",
        description="What to avoid in generation")
    model: InpaintingModel = Field(
        default=InpaintingModel.SD_INPAINTING)
    guidance_scale: float = Field(
        default=7.5, ge=1.0, le=20.0)
    num_inference_steps: int = Field(
        default=30, ge=10, le=100)
    seed: Optional[int] = Field(
        default=None, ge=0, le=2147483647)
    strength: float = Field(
        default=1.0, ge=0.0, le=1.0)
```

Listing 3: Response model and inpainting request schema (`schemas/image.py`)

Listing 4 shows the restoration request schema and its supporting enums. The `FaceModel` enum selects between CodeFormer and GFPGAN, while `UpscaleOption` controls the Real-ESRGAN scale factor. Boolean flags enable or disable individual pipeline stages.

```
class FaceModel(str, Enum):
    CODEFORMER = "codeformer"
    GFPGAN = "gfpgan"

class UpscaleOption(str, Enum):
    NONE = "none"
    UPSCALE_2X = "2x"
    UPSCALE_4X = "4x"

class RestorationRequest(BaseModel):
    image: str = Field(...,
        description="Base64 encoded input image")
    enable_face_enhance: bool = Field(
        default=True, description="Enable face "
        "enhancement (CodeFormer/GFPGAN)")
    face_model: FaceModel = Field(
        default=FaceModel.CODEFORMER)
    fidelity: float = Field(
        default=0.5, ge=0.0, le=1.0,
```

```
        description="CodeFormer fidelity "
        "(0=quality, 1=fidelity)")
    upscale: UpscaleOption = Field(
        default=UpscaleOption.UPSCALE_2X)
    enable_scratch_removal: bool = Field(
        default=False)
    enable_colorize: bool = Field(
        default=False)
```

Listing 4: Restoration request schema and enums (`schemas/image.py`)

Table 20 summarises the complete API surface.

Table 20: API endpoint summary

Endpoint	Method	Description
`/api/inpainting/`	POST	Inpaint masked region with a text prompt
`/api/inpainting/models`	GET	List available inpainting models and VRAM requirements
`/api/style/`	POST	Apply style transfer with a preset or custom prompt
`/api/style/presets`	GET	List available style presets
`/api/restoration/`	POST	Run restoration pipeline (face, upscale, scratch, colour)
`/api/restoration/models`	GET	List available restoration models
`/api/health`	GET	Health check with GPU info and loaded pipelines

6.1.3 Inpainting Service

The inpainting feature supports four model families—Stable Diffusion 1.5, SDXL, Kandinsky 2.2, and FLUX.1 Fill—each available in multiple quantisation variants. Rather than hard-coding pipeline parameters, the service uses a declarative configuration dictionary (Listing 5) where each entry specifies the Hugging Face model ID, pipeline class name, data type, and optional quantisation or CPU-offload flags. This design makes adding new models a one-line dictionary entry rather than a code change.

```
INPAINTING_MODELS = {
    "sd-inpainting": {
        "model_id": "runwayml/"
                    "stable-diffusion-inpainting",
        "pipeline": "StableDiffusionInpaintPipeline",
```

```
        "torch_dtype": "float16",
    },
    "sdxl-inpainting": {
        "model_id": "diffusers/stable-diffusion-"
                    "xl-1.0-inpainting-0.1",
        "pipeline":
            "StableDiffusionXLInpaintPipeline",
        "torch_dtype": "float16",
    },
    "sdxl-inpainting-4bit": {
        "model_id": "diffusers/stable-diffusion-"
                    "xl-1.0-inpainting-0.1",
        "pipeline":
            "StableDiffusionXLInpaintPipeline",
        "torch_dtype": "float16",
        "quantization": "4bit",  # ~4 GB VRAM
    },
    "kandinsky-inpainting": {
        "model_id": "kandinsky-community/"
                    "kandinsky-2-2-decoder-inpaint",
        "pipeline": "AutoPipelineForInpainting",
        "torch_dtype": "float16",
    },
    "flux-fill-nf4": {
        "model_id": "black-forest-labs/"
                    "FLUX.1-Fill-dev",
        "pipeline": "FluxFillPipeline",
        "torch_dtype": "float16",
        "quantization": "nf4",  # ~10 GB VRAM
    },
    # ... additional variants omitted
}
```

Listing 5: Declarative model configuration dictionary (excerpt from `diffusion.py`)

The `inpaint()` method (Listing 6) implements the full request flow. It decodes the base64 image and mask, records the original dimensions, resizes inputs to the model's optimal resolution (512×512 for SD 1.5, 1024×1024 for SDXL and FLUX), loads the pipeline via the caching mechanism described in Section 6.1.6, and runs inference under `torch.inference_mode()` to disable gradient tracking. The result is resized back to the original dimensions and returned as a base64 string. All parameters—guidance scale, inference steps, seed, strength, and padding mask crop—are forwarded from the validated request schema.

```
def inpaint(self, image_b64, mask_b64, prompt,
            model_key="sd-inpainting", **kwargs):
    start_time = time.time()
    try:
        config = INPAINTING_MODELS[model_key]
        image, fmt = self.base64_to_image(image_b64)
        mask = self.base64_to_mask(mask_b64)
        original_size = image.size

        # Resize to model-optimal dimensions
        if "sdxl" in model_key \
                or "flux" in model_key:
            target_size = (1024, 1024)
        else:
            target_size = (512, 512)
        image = image.resize(target_size,
            Image.Resampling.LANCZOS)
        mask = mask.resize(target_size,
            Image.Resampling.LANCZOS)

        pipe = self._load_pipeline(
            model_key, config)

        pipe_kwargs = {
            "prompt": prompt,
            "image": image,
            "mask_image": mask,
            "guidance_scale":
                kwargs.get("guidance_scale", 7.5),
            "num_inference_steps":
                kwargs.get("num_inference_steps", 30),
            "strength":
                kwargs.get("strength", 1.0),
        }

        seed = kwargs.get("seed")
        if seed is not None:
            pipe_kwargs["generator"] = \
                torch.Generator(
                    device=self.device
                ).manual_seed(seed)
```

```
42
43          with torch.inference_mode():
44              result = pipe(**pipe_kwargs).images[0]
45
46          if result.size != original_size:
47              result = result.resize(original_size,
48                  Image.Resampling.LANCZOS)
49
50          result_b64 = self.image_to_base64(
51              result, fmt)
52          return result_b64, None, \
53              time.time() - start_time
54      except Exception as e:
55          return None, str(e), \
56              time.time() - start_time
```

Listing 6: Inpainting inference method (`diffusion.py`)

Listing 7 shows the router endpoint that connects the HTTP layer to the service. The router obtains the singleton `DiffusionService`, unpacks the validated request fields, and wraps the result in a uniform `ImageResponse`.

```
1  from fastapi import APIRouter
2  from app.schemas.image import (
3      InpaintingRequest, ImageResponse)
4  from app.services.diffusion import (
5      get_diffusion_service)
6
7  router = APIRouter()
8
9  @router.post("/", response_model=ImageResponse)
10 async def inpaint_image(
11     request: InpaintingRequest,
12 ) -> ImageResponse:
13     service = get_diffusion_service()
14     result_b64, error, processing_time = \
15         service.inpaint(
16             image_b64=request.image,
17             mask_b64=request.mask,
18             prompt=request.prompt,
19             model_key=request.model.value,
20             negative_prompt=request.negative_prompt,
21             guidance_scale=request.guidance_scale,
```

```
            num_inference_steps=
                request.num_inference_steps,
            seed=request.seed,
            strength=request.strength,
            padding_mask_crop=
                request.padding_mask_crop,
        )
    if error:
        return ImageResponse(success=False,
            error=error,
            model_used=request.model.value,
            processing_time=processing_time)
    return ImageResponse(success=True,
        image=result_b64,
        model_used=request.model.value,
        processing_time=processing_time)
```

Listing 7: Inpainting router endpoint (`routers/inpainting.py`)

6.1.4 Style Transfer Service

Style transfer is implemented using diffusion-based image-to-image (img2img) generation rather than traditional neural style transfer (Gatys et al., 2016). This approach leverages the same Stable Diffusion and SDXL models used for inpainting, but in img2img mode: the input image is partially noised according to a `strength` parameter and then denoised with a style-describing prompt. Higher strength values produce more stylised output at the cost of reduced fidelity to the original image.

The system provides nine predefined style presets (Listing 8), each mapping a user-friendly name to an optimised prompt string. The `STYLE_PROMPTS` dictionary serves as the preset library, while `STYLE_MODELS` mirrors the structure of `INPAINTING_MODELS` for img2img pipelines. Users may also supply a custom text prompt via the "custom" preset option.

```
STYLE_PROMPTS = {
    "oil_painting": "oil␣painting␣style,␣thick␣"
        "brushstrokes,␣vibrant␣colors,␣masterpiece",
    "watercolor": "watercolor␣painting␣style,␣"
        "soft␣edges,␣flowing␣colors,␣delicate",
    "anime": "anime␣style,␣cel␣shading,␣vibrant␣"
        "colors,␣japanese␣animation,␣detailed",
    "sketch": "pencil␣sketch,␣black␣and␣white,␣"
        "detailed␣linework,␣hand␣drawn",
    "cyberpunk": "cyberpunk␣style,␣neon␣lights,␣"
```

```
        "futuristic, dark atmosphere, sci-fi",
    "ghibli": "studio ghibli style, soft colors, "
        "whimsical, animated, magical",
    "pixel_art": "pixel art style, 8-bit, retro "
        "video game aesthetic, nostalgic",
    "pop_art": "pop art style, bold colors, comic "
        "book, andy warhol inspired",
    "impressionist": "impressionist painting, monet "
        "style, soft brushstrokes, light and color",
}

STYLE_MODELS = {
    "sdxl-img2img": {
        "model_id": "stabilityai/"
            "stable-diffusion-xl-base-1.0",
        "pipeline":
            "StableDiffusionXLImg2ImgPipeline",
        "torch_dtype": "float16",
    },
    "sdxl-img2img-4bit": {
        "model_id": "stabilityai/"
            "stable-diffusion-xl-base-1.0",
        "pipeline":
            "StableDiffusionXLImg2ImgPipeline",
        "torch_dtype": "float16",
        "quantization": "4bit",
    },
    "sd-img2img": {
        "model_id": "runwayml/"
            "stable-diffusion-v1-5",
        "pipeline":
            "StableDiffusionImg2ImgPipeline",
        "torch_dtype": "float16",
    },
}
```

Listing 8: Style presets and model configuration (`diffusion.py`)

The `style_transfer()` method (Listing 9) follows the same pattern as inpainting: decode, resize, load pipeline, build keyword arguments, run inference, resize back, and encode. The key difference is that it looks up the style prompt from `STYLE_PROMPTS` (falling back to the raw string for custom styles) and passes `strength` as the denoising strength.

```
def style_transfer(self, image_b64, style,
        model_key="sdxl-img2img", strength=0.6,
        **kwargs):
    start_time = time.time()
    try:
        image, fmt = self.base64_to_image(
            image_b64)
        original_size = image.size

        if "sdxl" in model_key:
            target_size = (1024, 1024)
        else:
            target_size = (512, 512)
        image = image.resize(target_size,
            Image.Resampling.LANCZOS)

        # Resolve style preset or use custom text
        style_prompt = STYLE_PROMPTS.get(
            style, style)

        config = STYLE_MODELS[model_key]
        pipe = self._load_pipeline(
            f"style-{model_key}", config)

        pipe_kwargs = {
            "prompt": style_prompt,
            "image": image,
            "strength": strength,
            "guidance_scale":
                kwargs.get("guidance_scale", 7.5),
            "num_inference_steps":
                kwargs.get("num_inference_steps",
                           30),
        }

        with torch.inference_mode():
            result = pipe(**pipe_kwargs).images[0]

        if result.size != original_size:
            result = result.resize(original_size,
                Image.Resampling.LANCZOS)
```

```
42
43          return self.image_to_base64(result, fmt),\
44              None, time.time() - start_time
45      except Exception as e:
46          return None, str(e), \
47              time.time() - start_time
```

Listing 9: Style transfer inference method (`diffusion.py`)

6.1.5 Restoration Service

The restoration feature uses specialised computer vision models rather than diffusion models. It supports four operations that can be combined in a single request: scratch removal, face enhancement (CodeFormer or GFPGAN), colourisation, and super-resolution upscaling (Real-ESRGAN). The pipeline ordering is deliberate: scratches are removed first to give face detection a cleaner input, faces are enhanced next, colourisation follows (operating on the enhanced image), and upscaling is applied last to avoid processing unnecessarily large images in earlier stages.

Listing 10 shows the main `restore()` method that orchestrates this pipeline. Each stage is gated by a boolean flag from the request, and the intermediate result is passed through sequentially.

```
1  def restore(self, image_b64,
2          enable_face_enhance=True,
3          face_model="codeformer",
4          fidelity=0.5, upscale="2x",
5          enable_scratch_removal=False,
6          enable_colorize=False):
7      start_time = time.time()
8      try:
9          image, fmt = self.base64_to_image(
10             image_b64)
11         result = image
12
13         # 1. Scratch removal (clean first)
14         if enable_scratch_removal:
15             result = self.remove_scratches(result)
16
17         # 2. Face enhancement
18         if enable_face_enhance:
19             if face_model == "codeformer":
20                 result = \
```

```
                    self.enhance_faces_codeformer(
                        result, fidelity)
            else:
                result = \
                    self.enhance_faces_gfpgan(result)

        # 3. Colourisation
        if enable_colorize:
            result = self.colorize(result)

        # 4. Upscaling (last to preserve quality)
        if upscale != "none":
            scale = 4 if upscale == "4x" else 2
            result = self.upscale(result, scale)

        result_b64 = self.image_to_base64(
            result, fmt)
        return result_b64, None, \
            time.time() - start_time
    except Exception as e:
        return None, str(e), \
            time.time() - start_time
```

Listing 10: Restoration pipeline orchestration (`restoration.py`)

Listing 11 shows the CodeFormer face enhancement implementation. The method uses a `FaceRestoreHelper` utility to detect faces via RetinaFace, align and warp each face to a canonical 512×512 crop, process it through the CodeFormer transformer model with the fidelity parameter `w`, and then paste the restored face back into the original image using an inverse affine transform. The `fidelity` parameter controls the trade-off between enhancement quality (`w`=0) and faithfulness to the original face (`w`=1).

```
def enhance_faces_codeformer(self, image, fidelity=0.5):
    import torch
    codeformer = self._load_codeformer(fidelity)
    model = codeformer['model']
    face_helper = codeformer['face_helper']
    img2tensor = codeformer['img2tensor']
    tensor2img = codeformer['tensor2img']

    img = self.pil_to_cv2(image)
    face_helper.clean_all()
    face_helper.read_image(img)
```

```
    face_helper.get_face_landmarks_5(
        only_center_face=False)
    face_helper.align_warp_face()

    for idx, cropped_face in enumerate(
            face_helper.cropped_faces):
        cropped_face_t = img2tensor(
            cropped_face / 255.,
            bgr2rgb=True, float32=True)
        cropped_face_t = cropped_face_t \
            .unsqueeze(0).to(self.device)
        try:
            with torch.no_grad():
                output = model(cropped_face_t,
                    w=fidelity, adain=True)[0]
                restored_face = tensor2img(
                    output.squeeze(0),
                    rgb2bgr=True,
                    min_max=(-1, 1))
            del output
        except RuntimeError:
            restored_face = cropped_face
        restored_face = \
            restored_face.astype('uint8')
        face_helper.add_restored_face(
            restored_face)

    face_helper.get_inverse_affine(None)
    restored_img = \
        face_helper.paste_faces_to_input_image()
    return self.cv2_to_pil(restored_img)
```

Listing 11: CodeFormer face enhancement (`restoration.py`)

6.1.6 Model Management and VRAM Optimization

Running multiple large diffusion models on consumer GPUs (8–16 GB VRAM) requires careful memory management. The backend employs several strategies:

1. **Lazy loading with caching.** Models are loaded into GPU memory only on first use and cached in a `_pipelines` dictionary. Subsequent requests for the same model reuse the cached pipeline, avoiding repeated downloads and weight loading.

2. **Quantisation.** For SDXL and FLUX models, the system supports 8-bit and 4-bit quantisation via BitsAndBytes (Dettmers et al., 2022). This reduces VRAM usage significantly: SDXL drops from 10–12 GB to approximately 4 GB at 4-bit precision, and FLUX.1 Fill drops from 22–24 GB to approximately 10 GB at NF4 precision.

3. **Attention slicing.** All pipelines enable attention slicing via `enable_attention_slicing()`, which computes attention in sequential chunks rather than all at once, trading speed for lower peak memory.

4. **CPU offloading.** For FLUX.1 Fill at full precision, the service uses `enable_model_cpu_offload()`, which keeps model weights on the CPU and transfers them to the GPU only during the forward pass of each component.

5. **Inference mode.** All inference runs under `torch.inference_mode()`, which disables gradient computation and autograd bookkeeping, reducing memory overhead.

Listing 12 shows the FLUX.1 Fill quantisation loader. Because FLUX has two large components—the transformer and the T5 text encoder—both must be quantised separately using their respective BitsAndBytesConfig classes (from `diffusers` and `transformers`). The quantised components are then passed to `FluxFillPipeline.from_pretrained()` with `device_map="auto"` for automatic device placement.

```
def _load_flux_fill_quantized(self, config,
        quantization, torch_dtype):
    from diffusers import FluxFillPipeline, AutoModel
    from diffusers import BitsAndBytesConfig \
        as DiffusersBnBConfig
    from transformers import T5EncoderModel
    from transformers import BitsAndBytesConfig \
        as TransformersBnBConfig

    model_id = config["model_id"]

    if quantization == "8bit":
        diff_qc = DiffusersBnBConfig(
            load_in_8bit=True)
        trans_qc = TransformersBnBConfig(
            load_in_8bit=True)
    else:  # 4bit or nf4
        quant_type = "nf4" \
            if quantization == "nf4" else "fp4"
        diff_qc = DiffusersBnBConfig(
            load_in_4bit=True,
```

```
            bnb_4bit_quant_type=quant_type,
            bnb_4bit_compute_dtype=torch_dtype)
        trans_qc = TransformersBnBConfig(
            load_in_4bit=True,
            bnb_4bit_quant_type=quant_type,
            bnb_4bit_compute_dtype=torch_dtype)

    transformer = AutoModel.from_pretrained(
        model_id, subfolder="transformer",
        quantization_config=diff_qc,
        torch_dtype=torch_dtype)

    text_encoder_2 = T5EncoderModel.from_pretrained(
        model_id, subfolder="text_encoder_2",
        quantization_config=trans_qc,
        torch_dtype=torch_dtype)

    pipe = FluxFillPipeline.from_pretrained(
        model_id, transformer=transformer,
        text_encoder_2=text_encoder_2,
        torch_dtype=torch_dtype,
        device_map="auto")
    return pipe
```

Listing 12: FLUX.1 Fill quantised loading (`diffusion.py`)

Listing 13 shows the general pipeline loader, which handles three cases: FLUX quantisation (delegated to the method above), SD/SDXL quantisation via `PipelineQuantizationConfig` (which automatically targets the UNet component), and standard full-precision loading. For non-quantised large models, CPU offloading is enabled when the configuration flag is set.

```
def _load_pipeline(self, pipeline_key, config):
    if pipeline_key in self._pipelines:
        return self._pipelines[pipeline_key]

    torch_dtype = self._get_torch_dtype(
        config["torch_dtype"])
    quantization = config.get("quantization")
    use_cpu_offload = config.get(
        "use_cpu_offload", False)

    if quantization and \
            config["pipeline"] == "FluxFillPipeline":
```

```
        pipe = self._load_flux_fill_quantized(
            config, quantization, torch_dtype)
    elif quantization:
        # SD/SDXL: PipelineQuantizationConfig
        from diffusers.quantizers import \
            PipelineQuantizationConfig
        pipeline_quant_config = \
            PipelineQuantizationConfig(
                quant_backend=
                    f"bitsandbytes_{quantization}",
                quant_kwargs={
                    f"load_in_{quantization}": True},
                components_to_quantize=["unet"])
        pipe = pipeline_class.from_pretrained(
            config["model_id"],
            torch_dtype=torch_dtype,
            quantization_config=
                pipeline_quant_config)
        pipe = pipe.to(self.device)
    else:
        pipe = pipeline_class.from_pretrained(
            config["model_id"],
            torch_dtype=torch_dtype)
        if use_cpu_offload:
            pipe.enable_model_cpu_offload()
        else:
            pipe = pipe.to(self.device)

    if hasattr(pipe, "enable_attention_slicing"):
        pipe.enable_attention_slicing()

    self._pipelines[pipeline_key] = pipe
    return pipe
```

Listing 13: General pipeline loading with caching (`diffusion.py`)

Table 21 summarises the VRAM requirements for each model at different precision levels.

Table 21: VRAM requirements by model and precision

Model	FP16/BF16	8-bit	4-bit
SD 1.5 Inpainting	5–7 GB	—	—
SDXL Inpainting	10–12 GB	∼6 GB	∼4 GB
Kandinsky 2.2	6–8 GB	—	—
FLUX.1 Fill	22–24 GB	∼16 GB	∼10 GB
SDXL img2img	10–12 GB	∼6 GB	∼4 GB
SD 1.5 img2img	5–7 GB	—	—
CodeFormer	2–4 GB	—	—
GFPGAN	2–4 GB	—	—
Real-ESRGAN	2–6 GB	—	—

6.2 Frontend Development

The frontend is a single-page application (SPA) built with React 19, TypeScript, and Tailwind CSS, bundled by Vite. It communicates with the backend entirely through JSON REST calls.

6.2.1 Project Structure

```
frontend/
  src/
    api/
      config.ts            # API URL, base64 utils
      imageApi.ts          # API client functions
      index.ts             # Barrel exports
    components/
      HomeTab.tsx           # Landing page
      InpaintingTab.tsx     # Inpainting workflow
      StyleTransferTab.tsx  # Style transfer workflow
      RestorationTab.tsx    # Restoration workflow
      MaskCanvas.tsx        # Canvas mask drawing
    App.tsx                 # Tab navigation shell
    main.tsx                # React entry point
  .env.example              # VITE_API_URL template
  package.json
  tailwind.config.js
  vite.config.ts
```

Listing 14: Frontend directory structure

Table 22 lists the key frontend dependencies.

Table 22: Key frontend dependencies

Package	Version	Purpose
React	19.0	UI component library
TypeScript	5.6	Static type checking
Vite	6.0	Build tool and dev server
Tailwind CSS	4.0	Utility-first CSS framework

6.2.2 User Interface Design

The application uses a tab-based navigation pattern with a shared header. The dark theme (Tailwind `slate-900` background with `indigo-600` accent) provides comfortable viewing during extended editing sessions and reduces visual distraction from the images being edited. Each feature tab follows a consistent three-step workflow: upload an image, configure parameters, and generate a result. Common UI patterns—error banners, loading spinners, before/after comparison, and download buttons—are repeated across all tabs for consistency.

Listing 15 shows the complete `App.tsx` component. The tab configuration is stored as a typed array, making it straightforward to add new tabs. Active tab state is managed with React's `useState` hook, and conditional rendering displays the appropriate feature component. The wireframes in Section 5.3 illustrate the layout of each tab.

```
import { useState } from 'react'
import HomeTab from './components/HomeTab'
import InpaintingTab
  from './components/InpaintingTab'
import StyleTransferTab
  from './components/StyleTransferTab'
import RestorationTab
  from './components/RestorationTab'

type Tab = 'home' | 'inpainting'
  | 'style-transfer' | 'restoration'

function App() {
  const [activeTab, setActiveTab] =
    useState<Tab>('home')

  const tabs: { id: Tab; label: string }[] = [
    { id: 'home', label: 'Home' },
```

```
    { id: 'inpainting', label: 'Inpainting' },
    { id: 'style-transfer',
      label: 'Style Transfer' },
    { id: 'restoration',
      label: 'Restoration' },
  ]

  return (
    <div className="min-h-screen bg-slate-900">
      <header className="border-b
          border-slate-700 p-6">
        <h1 className="text-3xl font-bold
            text-center">DiffusionDesk</h1>
      </header>
      <nav className="border-b border-slate-700">
        <div className="flex gap-1 p-2
            max-w-4xl mx-auto">
          {tabs.map((tab) => (
            <button key={tab.id}
              onClick={() => setActiveTab(tab.id)}
              className={activeTab === tab.id
                ? 'bg-indigo-600 text-white'
                : 'text-slate-400'}>
              {tab.label}
            </button>
          ))}
        </div>
      </nav>
      <main className="p-6 max-w-4xl mx-auto">
        {activeTab === 'home'
          && <HomeTab />}
        {activeTab === 'inpainting'
          && <InpaintingTab />}
        {activeTab === 'style-transfer'
          && <StyleTransferTab />}
        {activeTab === 'restoration'
          && <RestorationTab />}
      </main>
    </div>
  )
}
```

Listing 15: Main application shell (`App.tsx`)

6.2.3 Canvas and Mask Drawing

The inpainting feature requires users to paint a mask over the regions they want to edit. The `MaskCanvas` component implements this using a dual-element architecture: an `<img>` element displays the source image, and a `<canvas>` element is positioned absolutely on top as a transparent overlay. This separation ensures that the mask drawing does not modify the original image data.

A key challenge is coordinate scaling: the canvas has pixel dimensions matching the original image (e.g., 1920×1080), but is displayed at the container's CSS width (e.g., 800 pixels wide). The `canvasScale` factor (container width / image width) maps mouse/touch coordinates from display space to canvas space. This scale is recalculated on window resize.

Listing 16 shows the `draw()` function. The brush tool uses `source-over` compositing to paint semi-transparent red strokes; the eraser uses `destination-out` compositing, which removes pixels from the canvas (effectively erasing previously painted mask regions). Lines are drawn between consecutive positions to produce smooth strokes.

```
const draw = useCallback((x: number, y: number) => {
  const canvas = canvasRef.current
  const ctx = canvas?.getContext('2d')
  if (!ctx) return

  ctx.beginPath()
  ctx.lineCap = 'round'
  ctx.lineJoin = 'round'
  ctx.lineWidth = brushSize

  if (tool === 'brush') {
    ctx.globalCompositeOperation = 'source-over'
    ctx.strokeStyle = 'rgba(255, 0, 0, 0.5)'
  } else {
    // Eraser: remove pixels
    ctx.globalCompositeOperation =
      'destination-out'
    ctx.strokeStyle = 'rgba(0,0,0,1)'
  }

  if (lastPosRef.current) {
    ctx.moveTo(lastPosRef.current.x,
```

```
      lastPosRef.current.y)
    ctx.lineTo(x, y)
    ctx.stroke()
  } else {
    ctx.arc(x, y, brushSize / 2,
      0, Math.PI * 2)
    ctx.fill()
  }

  lastPosRef.current = { x, y }
}, [tool, brushSize])
```

Listing 16: Canvas drawing function (`MaskCanvas.tsx`)

When the user finishes drawing, the mask must be exported as a black-and-white PNG for the backend (white pixels indicate regions to inpaint). Listing 17 shows the `exportMask()` function, which creates a temporary canvas, fills it with black, reads the alpha channel of the drawing canvas, and sets any pixel with alpha > 0 to white. The result is exported as a data URL.

```
const exportMask = useCallback(() => {
  const canvas = canvasRef.current
  if (!canvas) return null

  const maskCanvas =
    document.createElement('canvas')
  maskCanvas.width = canvas.width
  maskCanvas.height = canvas.height
  const maskCtx = maskCanvas.getContext('2d')
  if (!maskCtx) return null

  // Black background (areas to keep)
  maskCtx.fillStyle = 'black'
  maskCtx.fillRect(0, 0,
    maskCanvas.width, maskCanvas.height)

  const ctx = canvas.getContext('2d')
  if (!ctx) return null

  const imageData = ctx.getImageData(
    0, 0, canvas.width, canvas.height)
  const maskData = maskCtx.getImageData(
    0, 0, maskCanvas.width, maskCanvas.height)
```

```
24
25    // Alpha > 0 becomes white (inpaint region)
26    for (let i = 0;
27         i < imageData.data.length; i += 4) {
28      if (imageData.data[i + 3] > 0) {
29        maskData.data[i] = 255     // R
30        maskData.data[i + 1] = 255 // G
31        maskData.data[i + 2] = 255 // B
32        maskData.data[i + 3] = 255 // A
33      }
34    }
35
36    maskCtx.putImageData(maskData, 0, 0)
37    return maskCanvas.toDataURL('image/png')
38  }, [])
```

Listing 17: Mask export function (`MaskCanvas.tsx`)

6.2.4 API Integration

The frontend communicates with the backend through a thin API client layer in `src/api/`. Listing 18 shows the configuration module, which reads the backend URL from an environment variable (`VITE_API_URL`) set at build time by Vite. This allows the same codebase to target a local development server or a remote Colab instance by changing a single `.env` file. The module also provides utility functions for converting between `File` objects and base64 strings.

```
1  export const API_BASE_URL =
2    import.meta.env.VITE_API_URL
3    || 'http://localhost:8000';
4
5  export async function fileToBase64(
6      file: File): Promise<string> {
7    return new Promise((resolve, reject) => {
8      const reader = new FileReader();
9      reader.readAsDataURL(file);
10     reader.onload = () => {
11       resolve(reader.result as string);
12     };
13     reader.onerror = (error) => reject(error);
14   });
15 }
16
```

```
export function base64ToDataUrl(
    base64: string,
    mimeType: string = 'image/png'): string {
  if (base64.startsWith('data:'))
    return base64;
  return `data:${mimeType};base64,${base64}`;
}
```

Listing 18: API configuration (`config.ts`)

Listing 19 shows the `inpaintImage()` API function. It converts the uploaded `File` to base64, constructs a JSON body with snake_case keys (matching the Pydantic schema), sends a POST request, and converts the response image from base64 to a data URL for display in an `<img>` tag.

```
export async function inpaintImage(
    params: InpaintingParams
  ): Promise<ImageResponse> {
  const imageBase64 =
    await fileToBase64(params.image);

  const response = await fetch(
    `${API_BASE_URL}/api/inpainting/`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json' },
    body: JSON.stringify({
      image: imageBase64,
      mask: params.mask,
      prompt: params.prompt,
      negative_prompt: params.negativePrompt,
      model: params.model || 'sd-inpainting',
      guidance_scale:
        params.guidanceScale ?? 7.5,
      num_inference_steps:
        params.numInferenceSteps ?? 30,
      seed: params.seed ?? null,
      strength: params.strength ?? 1.0,
      padding_mask_crop:
        params.paddingMaskCrop ?? null,
    }),
  });

```

```
  if (!response.ok)
    throw new Error(
      `Inpainting failed: ${response.statusText}`
    );

  const data: ImageResponse =
    await response.json();
  if (data.success && data.image)
    data.image = base64ToDataUrl(data.image);
  return data;
}
```

Listing 19: Inpainting API client function (`imageApi.ts`)

Listing 20 shows the TypeScript interfaces that mirror the backend Pydantic schemas. The frontend uses camelCase naming conventions (e.g., `negativePrompt`), which are mapped to the backend's snake_case keys (e.g., `negative_prompt`) in the `JSON.stringify()` call.

```
export interface ImageResponse {
  success: boolean;
  image?: string;
  error?: string;
  model_used?: string;
  processing_time?: number;
}

export interface InpaintingParams {
  image: File;
  mask: string;  // base64 data URL
  prompt: string;
  negativePrompt?: string;
  model?: string;
  guidanceScale?: number;
  numInferenceSteps?: number;
  seed?: number | null;
  strength?: number;
  paddingMaskCrop?: number | null;
}

export interface StyleTransferParams {
  image: File;
  style: string;
```

```
  model?: string;
  strength?: number;
  negativePrompt?: string;
  guidanceScale?: number;
  numInferenceSteps?: number;
  seed?: number | null;
}

export interface RestorationParams {
  image: File;
  enableFaceEnhance?: boolean;
  faceModel?: 'codeformer' | 'gfpgan';
  fidelity?: number;
  upscale?: 'none' | '2x' | '4x';
  enableScratchRemoval?: boolean;
  enableColorize?: boolean;
}
```

Listing 20: TypeScript API interfaces (`imageApi.ts`)

7 Project Difficulties and Learning Outcomes

7.1 Project Difficulties

Several significant challenges were encountered during the development of DiffusionDesk. This subsection documents the key difficulties and the approaches taken to address them.

7.1.1 GPU Memory Constraints

The most persistent challenge throughout the project was managing GPU memory (VRAM). Diffusion models are inherently memory-intensive: Stable Diffusion XL requires 10–12 GB in FP16 precision, while FLUX.1 Fill demands 22–24 GB at full precision—exceeding the capacity of most consumer GPUs and the free-tier Google Colab environment (typically 15 GB with a T4 GPU).

Addressing this required implementing multiple optimisation strategies simultaneously. BitsAndBytes quantisation was adopted to reduce VRAM usage: 8-bit and 4-bit (NF4) quantisation for FLUX.1 Fill reduced requirements from 22–24 GB to approximately 16 GB and 10 GB respectively. For SDXL models, `PipelineQuantizationConfig` was used to selectively quantise the UNet component. Additionally, `enable_attention_slicing()` and `enable_model_cpu_offload()` were applied to further reduce peak memory consumption. Balancing output quality against memory usage required extensive experimentation, as aggressive quantisation can introduce visible artefacts.

7.1.2 Model Loading Latency

Diffusion models have large file sizes (several gigabytes each), and loading them from disk into GPU memory is slow. The first inference request for a given model could take 30–60 seconds as the model weights are loaded, quantised, and transferred to the GPU. This created a poor user experience, particularly when switching between models.

The solution was a lazy-loading strategy with an in-memory cache: once a model pipeline is loaded, it is stored in a dictionary keyed by model name and quantisation level. Subsequent requests for the same model return the cached pipeline immediately. While this approach trades memory for speed, it significantly improves the experience for repeated use of the same model.

7.1.3 Canvas Coordinate Scaling

The mask drawing feature required careful handling of coordinate systems. The HTML `<canvas>` element maintains pixel dimensions matching the original uploaded image (e.g., 1920×1080), but is displayed at a smaller CSS size to fit the browser viewport. Mouse and touch coordinates are reported in CSS pixels, not canvas pixels, creating a mismatch that causes the drawn mask to appear offset from the cursor.

Solving this required computing a `canvasScale` factor (the ratio of the canvas's internal width to its displayed CSS width) and multiplying all pointer coordinates by this factor before drawing. The scale must be recalculated whenever the window is resized, which was handled using a `ResizeObserver` callback.

7.1.4 Base64 Image Encoding

Transmitting images between the frontend and backend required choosing a serialisation format. Multipart form data is the conventional approach for file uploads, but it complicates the API design when additional parameters (prompt, model, guidance scale, etc.) must accompany each image. Instead, base64-encoded strings embedded in JSON request bodies were used, allowing all parameters to be sent in a single, structured payload.

This approach simplifies the API contract and makes requests self-contained, but increases payload size by approximately 33% due to base64 encoding overhead. For the image sizes processed by this application (typically under 5 MB), this trade-off was acceptable. However, the frontend required utility functions to convert between `File` objects, base64 strings, and data URLs, adding implementation complexity.

7.1.5 Cross-Origin Deployment

When deploying the backend on Google Colab with ngrok tunnelling, the frontend (hosted on `localhost` or Vercel) and backend (served through an ngrok URL) run on different origins. This triggers browser CORS (Cross-Origin Resource Sharing) restrictions, blocking API requests by default.

Configuring FastAPI's `CORSMiddleware` with `allow_origins=["*"]` resolved this for development, but required understanding the interaction between CORS preflight requests, the ngrok proxy layer, and the browser's same-origin policy. Debugging CORS errors was time-consuming, as the browser's error messages are often generic and do not clearly indicate which header or configuration is missing.

7.2 Learning Outcomes

This project provided substantial learning across multiple domains of software engineering and machine learning. The key learning outcomes are summarised below.

7.2.1 Diffusion Model Inference

Working directly with diffusion model pipelines through the Hugging Face Diffusers library provided practical understanding of how these models operate at inference time. Key concepts learned include the denoising process, the role of schedulers (e.g., DDIM, Euler) in controlling the number of inference steps, the effect of guidance scale on prompt adherence, and how

conditioning inputs (text prompts, mask images) guide the generation process. Understanding the distinction between latent-space diffusion (Stable Diffusion, SDXL) and pixel-space models, as well as the architectural differences between UNet-based and Transformer-based (DiT) diffusion models, deepened theoretical knowledge of the field.

7.2.2 Model Quantisation and Memory Optimisation

Implementing quantisation strategies provided hands-on experience with techniques that are increasingly important as model sizes grow. Learning to apply BitsAndBytes 4-bit (NF4) and 8-bit quantisation, understanding the difference between quantising individual model components (e.g., the UNet only) versus entire pipelines, and measuring the quality–memory trade-offs were valuable skills. This experience is directly applicable to deploying large models in resource-constrained environments.

7.2.3 Full-Stack Web Development

Building both the FastAPI backend and the React frontend provided end-to-end experience with modern web development. On the backend, this included designing RESTful APIs, defining Pydantic schemas for request validation, implementing service layers with singleton patterns, and managing asynchronous request handling. On the frontend, this involved working with React hooks (`useState`, `useCallback`, `useRef`), TypeScript interfaces, the HTML Canvas API for interactive drawing, and environment-based configuration for flexible deployment.

7.2.4 Image Processing Pipelines

The restoration feature required understanding how multiple image processing operations can be composed into a pipeline. Learning to coordinate face detection, alignment, enhancement (CodeFormer/GFPGAN), scratch removal, colourisation, and upscaling (Real-ESRGAN)—and understanding why the ordering of these operations matters for output quality—provided insight into practical computer vision system design.

7.2.5 Cloud GPU Deployment

Deploying the backend on Google Colab and exposing it via ngrok provided experience with cloud-based GPU computing, including environment setup, dependency management, and network tunnelling. Understanding the constraints of ephemeral cloud environments (session timeouts, limited disk space, variable GPU availability) and designing the application to work within these constraints was a practical lesson in deployment engineering.

8 Future Implementation

This section outlines planned enhancements and extensions that could improve DiffusionDesk beyond the current implementation. These are organised by priority and feasibility.

8.1 Text-to-Image Generation

The current application focuses on editing existing images (inpainting, style transfer, restoration). Adding a text-to-image generation tab would allow users to create images from scratch using text prompts. The backend infrastructure already supports diffusion model inference, so this extension would primarily require a new router endpoint, a frontend tab with prompt input fields, and integration with text-to-image pipelines such as Stable Diffusion XL or FLUX.1. This is a natural extension of the existing architecture and would significantly broaden the application's utility.

8.2 ControlNet Integration

ControlNet enables spatially-conditioned image generation by providing additional control signals such as edge maps, depth maps, or pose skeletons. Integrating ControlNet with the existing inpainting and style transfer features would give users finer control over the generated output—for example, preserving the structural composition of the original image while applying a style transfer. The Diffusers library provides pre-built ControlNet adapters that could be integrated with the existing pipeline loading infrastructure.

8.3 Batch Processing

The current system processes one image per request. Adding batch processing support would allow users to apply the same editing operation to multiple images simultaneously. This would be particularly useful for style transfer (applying a consistent style across a series of photographs) and restoration (enhancing a collection of old photographs). Implementation would require a queue-based backend architecture and a frontend interface for managing multiple uploads and tracking progress.

8.4 User Authentication and History

Adding user authentication would enable session persistence and editing history. Users could save their editing configurations, revisit previous results, and build on earlier work. This would require integrating an authentication provider (e.g., OAuth), adding a database layer for storing user data and processing history, and extending the frontend with login and history views.

8.5 Additional Model Support

The diffusion model ecosystem evolves rapidly, with new and improved models released frequently. The application's modular architecture—where model configurations are defined in dictionary structures and pipelines are loaded dynamically—facilitates adding new models. Potential additions include:

- **SDXL Turbo / SD Turbo** – Distilled models that generate results in 1–4 inference steps, enabling near-real-time editing.
- **Stable Diffusion 3** – The latest generation of Stable Diffusion with improved text rendering and image quality.
- **IP-Adapter** – Image prompt adapters that allow using a reference image (rather than text) to guide style transfer, enabling more intuitive style selection.

8.6 Persistent Deployment

The current deployment relies on ephemeral cloud environments (Google Colab) that require re-setup for each session. A persistent deployment on a dedicated GPU server or a cloud GPU service (e.g., AWS EC2 with GPU instances, RunPod, or Vast.ai) would provide a stable, always-available service. This would also enable features that require persistent storage, such as user accounts, editing history, and model caching across sessions.

Bibliography

Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. (2022). LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35.

Gatys, L. A., Ecker, A. S., and Bethge, M. (2016). Image style transfer using convolutional neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Grand View Research (2023). AI image generator market size, share & trends analysis report, 2024–2030. Accessed: 2026-02-02.

Ho, J., Jain, A., and Abbeel, P. (2020). Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Ho, J. and Salimans, T. (2022). Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*.

Ma, X., Zhou, X., Huang, H., Jia, G., Wang, Y., Chen, X., and Chen, C. (2023). Uncertainty-aware image inpainting with adaptive feedback network. *Expert Systems with Applications*, 235:121148.

Peebles, W. and Xie, S. (2023). Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. (2021). Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*.

Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Song, J., Meng, C., and Ermon, S. (2021). Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*.

Wang, X., Li, Y., Zhang, H., and Shan, Y. (2021a). GFP-GAN: Towards real-world blind face restoration with generative facial prior. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Wang, X., Xie, L., Dong, C., and Shan, Y. (2021b). Real-ESRGAN: Training real-world blind super-resolution with pure synthetic data. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*.

Zhou, S., Chan, K. C., Li, C., and Loy, C. C. (2022). Towards robust blind face restoration with codebook lookup transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*.